Databases project 2025-2026

Choose your adventure

Member list (name and email): Jan Francesc Porcar Guillen (janfracesc.porcar@students.salle.url.edu) , Francesc Mateu Vallespin (f.mateu@students.salle.url.edu), Giovanni Luca Giraldo Cabana (giovanniluca.giraldo@students.salle.url.edu)

Dataset: Environment

Delivery date: May 17th

Index

1 INTRODUCTION....2

1.1 Project team....2

1.2 Dataset description....2

1.3 Validation questions....3

2 ENTITY-RELATIONSHIP MODEL....4

2.1 Diagram....4

2.2 Justification....5

3 RELATIONAL MODEL....8

3.1 Diagram....8

3.2 Justification....9

4 DATATYPE SELECTION....11

5 PHYSICAL MODEL CODIFICATION....15

6 DATA IMPORT....17

6.1 File_1.csv....17

6.2 File_2.csv....18

6.3 File_3.csv....19

6.4 File_4.csv....19

6.5 File_5.csv....20

6.6 File_6.csv....20

7 QUERY IMPLEMENTATION....21

7.1 (Which power plants have been operational for more than 30 years?)....21

7.2 (Which companies own more than 3 power plants?)....24

7.3 (Which countries produce more energy than they consume in 2015?)....26

7.4 (Which energy type has the highest average CO_2 emission across all countries?)....28

7.5 (Which countries have increased their energy consumption every year between 2010 and 2020?)....30

7.6 (Which countries have a GDP above the world average GDP in 2015?)....34

7.7 (Which countries have more than 5 power plants?)....37

7.8 What is the average capacity (MW) of power plants grouped by type)....39

7.9 (Which power plants have an annual generation above the average generation of all plants?).....40

7.10 (Which cities are both economically expensive (Cost of Living Index > 80) and experience significant particulate pollution (Average PM2.5 > 30?)).....42

7.11 (Which countries have an energy intensity per capita above the world average in 2015?).....45

7.12 (Which countries have reduced their CO_2 emissions from electricity and heat between 2015 and 2019?).....46

7.13 (Which country has the highest CO_2 emissions from transport in 2015?).....50

7.14 (Which countries have CO_2 emissions from industry above the average of all countries in 2010?)...52

7.15 (What is the total CO_2 emission from all sectors per country in 2015?).....54

7.16 (Which countries have higher CO_2 emissions from electricity and heat than from transport in 2015?).....57

7.17 (Which city had the worst average PM2.5 level across all recorded dates?).....59

7.18 (Which countries have an average NO_2 level above 50 across all their cities?).....63

7.19 (Which cities have an average humidity above 80% and an average PM10 above 100?).....65

7.20 (Which city had the highest average temperature recorded in the air quality data?).....67

7.21 (Which city has the highest average temperature in July across all recorded years?).....69

7.22 (Which country has the highest average temperature in winter (December, January, February)?)....71

7.23 (Which cities have a cost of living index above the average cost of living index of all cities?).....73

7.24 (Which cities have a cost of living index above average and belong to countries with high NO_2 levels?).....75

7.25 (What are the top 5 cities with the highest local purchasing power index?).....77

7.26 (Which countries have more than 3 active government policies?).....79

7.27 (Which countries have more than 3 active government policies).....81

7.28 (Which countries were both top CO_2 emitters and had active emissions reduction policies in 2015?).....83

7.29 (Which countries appear as extreme cases in 2015, either very high GDP or very high CO_2 emissions?).....86

7.30 (Which countries have no government policy recorded?).....88

7.31 TRIGGER 1.....91
7.32 TRIGGER 2.....94
7.33 TRIGGER 3.....98

8 CONCLUSIONS.....101

8.1 UTILIZATION OF RESOURCES.....101
8.2 AI USAGE.....102
8.3 LESSONS LEARNED AND CONCLUSIONS.....107

1 Introduction

The main objective of this project is to design and implement a relational database from scratch, working in a team of three people. As AI & Data Science students, we wanted to work with a dataset that is not only interesting but also highly relevant to current global challenges. That is why we chose a dataset based on environmental data, focusing on historical climate trends, global energy consumption, and CO2 emissions. In the modern fight against climate change, data analysis is crucial. Every single day generates thousands of data points regarding temperature fluctuations across the globe, energy grid transitions, and the carbon footprint of different industries. Our motivation is to take these raw, messy CSV files and transform them into a fully normalized relational database (up to the third normal form). By doing this, we will be able to easily query the data to discover interesting patterns about global warming trends, the efficiency of green energy transitions, and how different regions are managing their environmental impact.

1.1 Project team

We distributed the work into the following:

- **Giovanni:** he handled raw data to perform the conceptual and relational models. In the implementation phase, he was in charge of coding the physical model to create the database in Oracle, importing his assigned CSV files, developing specific SQL queries with their respective validations, implementing database triggers, and redacting parts of the report.
- **Francesc:** he performed a part of the conceptual model and helped with the relational model. For the physical phase, he managed the execution of the database schema in Oracle, handled the data import and inserts via the Oracle wizard for his entities, developed a share of the advanced SQL queries and validations, coded triggers, and redacted parts of the report.
- **Jan:** he created the new CSV required and integrated it into the conceptual and relational models. During the physical implementation, he contributed to instantiating the database in Oracle, managed the CSV uploads for his respective tables, wrote complex SQL queries alongside their validations, assisted with trigger implementation, and redacted parts of the report.

1.2 Dataset description

The starting point for our project is a collection of CSV files containing global environmental, climate, and socioeconomic data. This is divided into several files that capture different aspects of the planet's health and human activity:

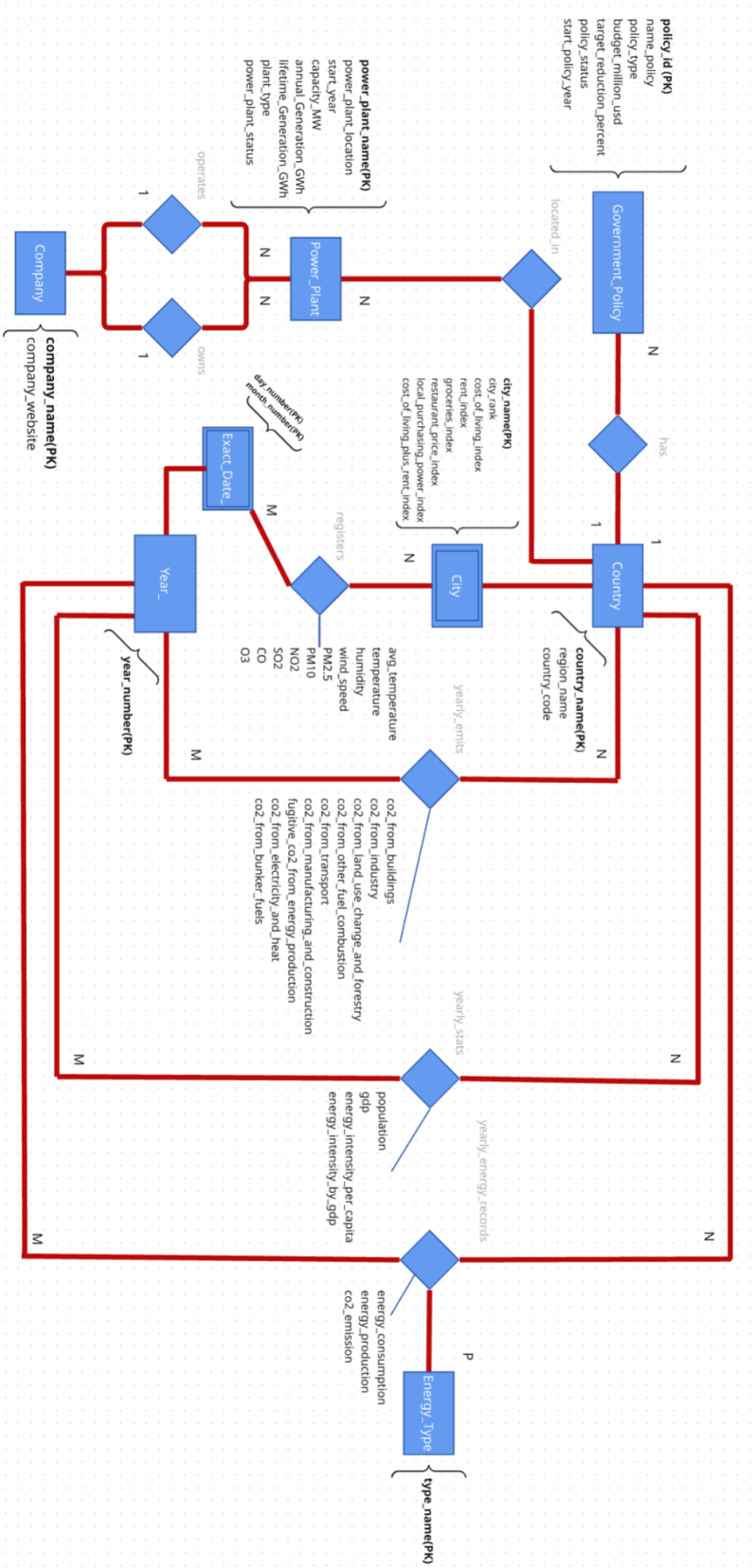
- city_temperature.csv: contains the historical weather records, including daily average temperatures for cities and countries all around the world over several years.
- energy.csv & nuclear_power_dataset.csv: these act as our energy tracking datasets, detailing global energy consumption, the transition between fossil fuels and renewable sources, and specific metrics regarding worldwide nuclear power generation.
- co-emissions-by-sector.csv & global_air_quality_data_10000.csv: these files store specific data regarding pollution. They detail the carbon footprint generated by different industrial sectors and track real-world air quality indices, allowing us to see the direct environmental impact of human activity.
- Cost_of_living_index.csv: provides crucial socioeconomic context, allowing us to eventually correlate environmental metrics (such as energy transitions or pollution levels) with the economic development and living costs of different regions.

1.3 Validation questions

1. Which power plants have been operational for more than 30 years?
2. Which companies own more than 3 power plants?
3.Which countries produce more energy than they consume in 2015?
4.Which energy type has the highest average CO_2 emission across all countries?
5.Which countries have increased their energy consumption every year between 2010 and 2020?
6.Which countries have a GDP above the world average GDP in 2015
7.Which countries have more than 5 power plants?
8.What is the average capacity (MW) of power plants grouped by type
9. Which power plants have an annual generation above the average generation of all plants?
10. Which cities are both economically expensive (Cost of Living Index > 80) and experience significant particulate pollution (Average PM2.5 > 30?)
11. Which countries have an energy intensity per capita above the world average in 2015?
12. Which countries have reduced their CO_2 emissions from electricity and heat between 2015 and 2019?
13. Which country has the highest CO_2 emissions from transport in 2015?
14. Which countries have CO_2 emissions from industry above the average of all countries in 2010?
15. What is the total CO_2 emission from all sectors per country in 2015?
16. Which countries have higher CO_2 emissions from electricity and heat than from transport in 2015?
17. Which city had the worst average PM2.5 level across all recorded dates?
18. Which countries have an average NO_2 level above 50 across all their cities?
19. Which cities have an average humidity above 80% and an average PM10 above 100?
20. Which city had the highest average temperature recorded in the air quality data?
21. Which city has the highest average temperature in July across all recorded years?
22. Which country has the highest average temperature in winter (December, January, February)?
23. Which cities have a cost of living index above the average cost of living index of all cities?
24. Which cities have a cost of living index above average and belong to countries with high NO_2 levels?
25. What are the top 5 cities with the highest local purchasing power index?
26. Which countries have more than 3 active government policies
27. Which countries have more than 3 active government policies
28. Which countries were both top CO_2 emitters and had active emissions reduction policies in 2015?
29. Which countries appear as extreme cases in 2015, either very high GDP or very high CO_2 emissions?
30. Which countries have no government policy recorded?

2 Entity-relationship model

2.1 Diagram



https://miro.com/welcomeonboard/NGdXY2hRTVpaZVhPVjYyelNSWlBCc3VvUUFvaStxbDF3eUh6WTNKdzRFendNZGFQajFRT0h1ZGlqZVlaK2ZvcmxjWW1ZRXRWWVJSSzR4ZVpUeXlpMVZ1aWVLZEhtK0JId1dqeTFzalZmMDM5YXJWQ05ZMTl2eitjd043V2lnSzJ0R2lncW1vRmFBVnlLcVJzTmdFdlNRPT0hdjE=?share_link_id=343155690679

2.2 Justification

The conceptual design of our global climate and energy database was guided by two main principles: accurately reflecting the complex geographical and temporal realities of the environmental data provided, and ensuring strict adherence to database normalization rules (up to the Third Normal Form). Below is a detailed explanation of the key design decisions made during the creation of the Entity-Relationship (ER) model.

1. The temporal spine: the split of Date into Year_ and Exact_Date_

One of the most critical and complex decisions in our model was elevating the time dimension from a simple attribute to a topological hierarchy of strong and weak entities. Instead of a single monolithic date entity, we engineered a structural split into a strong entity representing the year (Year_) and a weak entity for the exact date (Exact_Date_). This split was justified because environmental datasets operate on distinct temporal granularities. Macro-economic indicators and national emissions are strictly tracked annually, whereas meteorological data and air quality are tracked daily. Attempting to force annual data into a daily date structure violates data integrity. By creating Year_ as a strong, independent entity with the year number as its primary key, we established a clean, unified temporal spine for all macro-level relationships. We then modeled Exact_Date_ as a weak entity depending hierarchically on the year. Its composite identifier, consisting of the day, month, and year numbers, acts as the anchor for highly granular transactional data. This structural decomposition ensures absolute temporal accuracy, preventing normalization anomalies while maintaining the First Normal Form regarding atomicity.

2. Geographical hierarchy and the treatment of "Region"

The geographical modeling required a careful balance between detail and normalization, specifically regarding the Country and City entities. In our datasets, the Region column represents continental zones. We determined that for every instance of a country, there is only one corresponding region. Given this strict one-to-one mapping at the country level, modeling the region as an attribute of the Country entity is the most efficient solution, simplifying the architecture without any risk of data duplication. Furthermore, we modeled City as a weak entity because a city cannot be uniquely identified in a global context by its name alone. Therefore, the City depends on the Country for its identity, forming a composite identifier of city name and country name that ensures structural integrity.

3. Normalization of air quality and temperature: the "Registers" relationship

The Registers relationship, connecting City and Exact_Date_, is the primary vessel for observational data. The attributes attached to this relationship (e.g., avg_temperature, PM2.5, NO2, CO) originate from the merging of distinct air quality and daily temperature datasets. We decided to model these metrics as attributes of the associative relationship rather than the entity itself because a temperature reading or a pollution index is only valid when associated with a specific territory at a specific

moment in time. By placing these attributes in the relationship, we ensure they depend entirely on the massive composite key formed by the city, country, and exact date components, which prevents update anomalies. Additionally, during the analysis of the city_temperature.csv data, we discovered that the State column contained exclusively null values for all international records, so we omitted this attribute from the conceptual model to avoid database bloat.

4. Avoiding redundancy in global emissions: the "Yearly_Emits" relationship

The Yearly_Emits relationship handles the complex sector-based CO2 data. To maintain complete traceability with the raw data, we included all nine specific emission sectors as attributes of this relationship. By linking these attributes to the intersection of Country and Year_, we ensure that the sectoral breakdown is unique per year. This complies strictly with the Third Normal Form (3NF), as these values depend solely on the primary keys of the participating entities and not on each other.

5. Demographic isolation and the "Yearly_Stats" rombo

A major breakthrough in our normalization process was the isolation of the Yearly_Stats relationship. The source energy dataset repeated population and GDP values for every type of energy within the same country and year. To resolve this, we extracted population, GDP, and the intensity metrics into a separate associative entity connecting Country directly to Year_. This was justified because a country's GDP remains the same whether evaluating its wind production or coal consumption. By isolating these metrics, we prevent the multiplication of macroeconomic records, ensuring an absolute absence of duplicates.

6. The energy triple-cross: The "Yearly_Energy_Records" ternary relationship

To handle the most granular part of our energy data, we implemented a ternary relationship connecting Country, Year_, and Energy_Type. Metrics like energy consumption and production cannot be identified by country and year alone; the third dimension of the energy type is required to find a unique value. By creating an independent Energy_Type entity, we avoid hardcoding energy categories into column headers. This makes the database extensible, as adding a new energy type in the future only requires inserting a row into the Energy_Type table rather than redesigning the schema.

7. Nuclear infrastructure: the "Power_Plant" and "Company" ecosystem

The modeling of nuclear power plants introduced the need for more complex organizational structures. Each plant is modeled as a strong entity representing a physical asset with a unique name, grouping its technical data as direct attributes. We observed that the dataset distinguished between plant owners and operators. Instead of creating two identical corporate entities, we created a single Company entity and drew two distinct relationships representing ownership and operation. This reflects the real-world reality where one company might own a plant while another manages its daily operations, providing a highly normalized relational solution that maximizes data reuse.

8. Economic Context: integrating "Cost of Living"

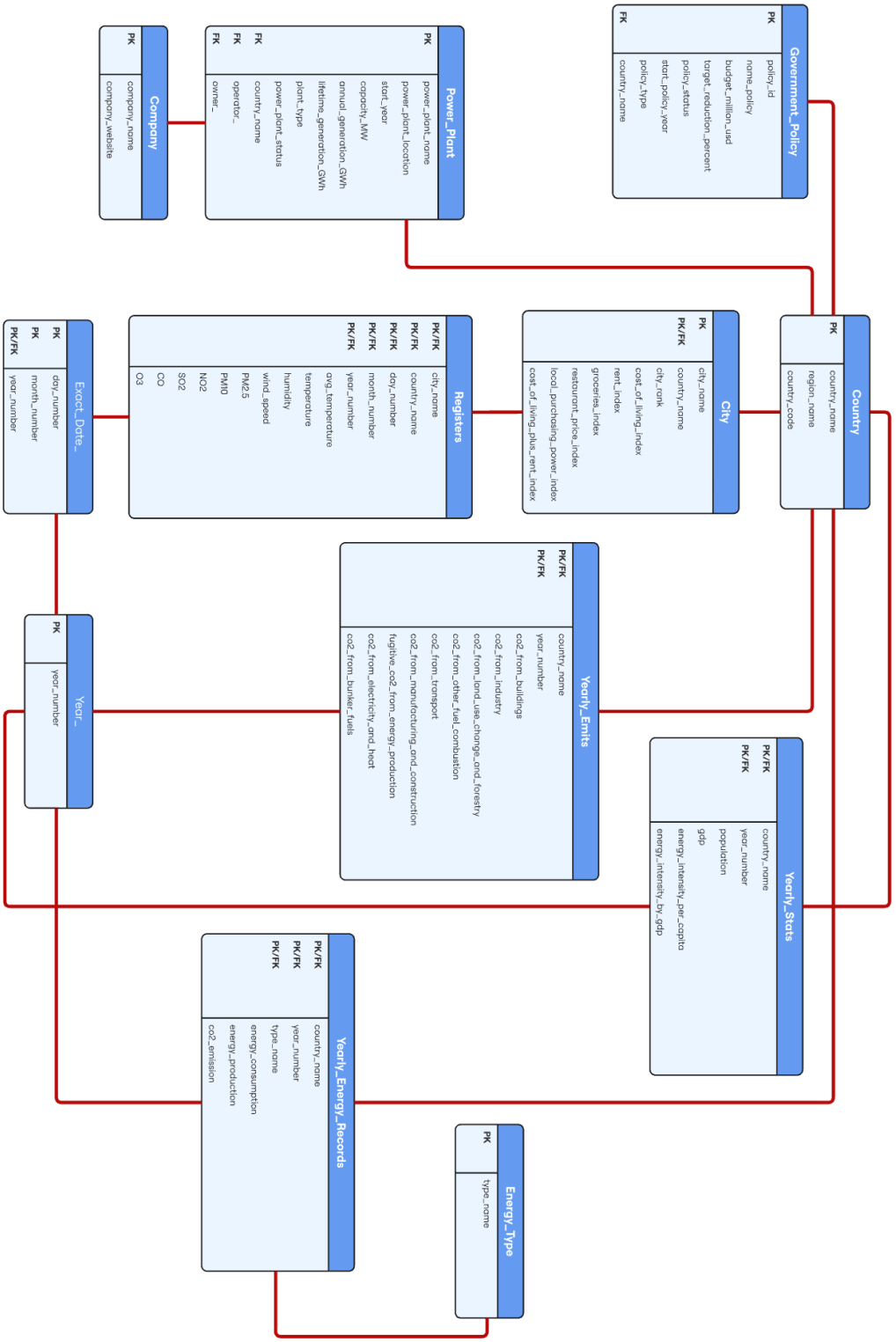
We integrated city-level economic data from the Cost_of_living_index.csv dataset. Unlike the climate data, indices for rent, groceries, and purchasing power do not have a temporal component in our dataset. Since these values depend solely on the municipality, they were added directly as attributes of the City entity, further justifying the necessity of modeling City as an identifiable hub for localized facts.

9. Government Policy: integrating the team-generated dataset

To enrich the analytical capabilities of our database, we created a new dataset to introduce a strong Government_Policy entity into our conceptual model. Each policy is uniquely identified by a surrogate policy identifier and contains attributes such as the policy name, type, budget, and target reduction percentages. These attributes depend solely on the policy identifier, ensuring compliance with the Third Normal Form. The connection to the existing model was established through a relationship linking Government_Policy to the Country entity via a 1:N cardinality. One country can have many government policies, but each policy belongs to exactly one country. This relationship was chosen because environmental policies are inherently tied to a specific nation.

3 Relational model

3.1 Diagram



https://miro.com/welcomeonboard/cXZ4TG9XazBCblY0U0pGOUJQa1VhaW8zME80Tm5TNWZJTlNFMDhJMHBCYlE3ZmVLbERFSzBMd0hKSFdEK2VmVTdiNTdTTWFwQlVvSnBIb0VmeFc5S2x1aWVLZEhtK0JId1dqeTFzalZmMDNkT1RVb21OS0JXZXNzYUMyaC94a3lQdGo1ZEV3bUdPQWRZUHQzSGl6V2NBPT0hdjE=?share_link_id=261672569553

3.2 Justification

The transition from the Conceptual Model (E-R) to the Relational Model was governed by formal transformation rules to ensure data consistency, minimize redundancy, and maintain the referential integrity of our global climate and energy database. This section justifies the structural changes and the resolution of complex relationships into a normalized table schema.

1. Resolution of N:M and ternary relationships into link tables

In accordance with relational theory, every associative and ternary relationship from our E-R diagram was transformed into an independent link table. For instance, the ternary relationship representing yearly energy records originally connected the country, the year, and the energy type. In the relational model, this was converted into a dedicated table where the primary key is a composite of three foreign keys: country name, year number, and type name. This strict mapping ensures that a specific energy metric is mathematically unique for each country, year, and fuel type intersection. Similarly, the many-to-many relationships managing sector-based emissions and daily registers were transformed into junction tables. The yearly emissions table utilizes a primary key composed of the country name and the year number. For the registers table, resolving the relationship between the weak city and exact date entities required a massive five-column composite primary key comprising the city name, country name, day, month, and year, allowing the database to store completely unique daily climate observations.

2. Implementation of weak entity City

The entity City was identified as a weak entity dependent on Country. During the transformation, the primary key of the strong entity (country_name) was migrated into the City table. To ensure global uniqueness, the primary key for the City table is defined as a composite of city_name and country_name. This prevents collisions between municipalities with identical names in different nations (e.g., London, UK vs. London, Canada) and maintains the identifying relationship established in the conceptual phase. The exact date entity was similarly modeled as weak and dependent on the isolated year entity, resulting in a composite primary key consisting of the day, month, and year numbers.

3. Management of 1:N relationships and Foreign Key placement

The transformation of one-to-many relationships required the propagation of the primary key from the parent side to the child side as a foreign key to establish the link between tables. The relationship locating power plants was resolved by adding the country name as a foreign key in the power plant table. This reflects that one country can host multiple plants, but each plant is physically tied to a single national jurisdiction. To accurately model the distinct corporate roles found in the nuclear dataset, the power plant table includes two separate foreign keys, owner and operator, both referencing the company name in the company table. This allows the database to distinguish between corporate ownership and daily management for each infrastructure asset.

4. Normalization and the omission of redundant attributes

The relational model was strictly refined to satisfy the Third Normal Form (3NF) by ensuring all non-key attributes are functionally dependent only on the primary key. A critical structural optimization was the separation of the yearly statistics table. Attributes such as population, GDP, and energy intensity indices were placed in their own relationship table rather than being included directly in the granular energy records. This prevents the mass duplication of demographic data, as these values do not vary by energy type. Additionally, groups of related attributes, such as those for

the cost of living indices or plant performance metrics, which were visually grouped in the E-R model, were flattened into individual atomic columns within their respective tables to satisfy the First Normal Form requirement for atomicity.

5. Referential Integrity and Data Types

Every connection in the relational model is supported by strict constraints. By defining foreign keys carefully, we mathematically guarantee that no climate register can exist without a corresponding city, and no energy record can exist without a valid energy type. This enforcement preserves the structural soundness of the entire schema across millions of rows.

6. Government_Policy: integration of the team-generated dataset

The government policy table was derived from the strong entity of the same name, introduced through the team-generated dataset created with Mockaroo. Since it is a strong entity with a simple, single-column surrogate primary key called the policy identifier, no junction table was required. The one-to-many relationship with the country was resolved by propagating the primary key of the parent side into the child side, following the same rule applied to power plants and countries. Specifically, the country name was added as a foreign key in the government policy table. This ensures that every policy is traceable to a specific nation while avoiding data duplication. The resulting table is fully compliant with the Third Normal Form: all non-key attributes (policy name, policy type, budget, target reduction, status, and year) depend solely and directly on the primary key, with no transitive dependencies present.

4 Datatype selection

The following section details the Oracle data type selected for every attribute of each physical model table. Type choices were first proposed individually by each team member for the tables under their responsibility, then reviewed and agreed upon collectively. The justification for each decision references: (1) the maximum length / value range observed in the source CSV data during the profiling phase, (2) the need to support NULL values where data is incomplete, and (3) relational algebra principles prioritizing JOIN performance and data structural safety.

Table: COUNTRY

Stores all sovereign entities referenced across every CSV source

- country_name (PK) [VARCHAR(255)]: Country names vary widely. To ensure maximum structural safety against truncation errors during bulk data loads, VARCHAR(255) was universally adopted for text primary keys.
- region_name [VARCHAR(255)]: Represents continental zones. VARCHAR(255) applied for uniformity. NULLable, since not all CSVs include macro-region data.
- country_code [VARCHAR(10)]: ISO codes from the emissions CSV. A strict VARCHAR(10) was chosen to optimize storage for these standardized short strings. NULLable.

Table: YEAR_

Extracted to act as a standalone temporal dimension for annual facts

- year_number (PK) [INT]: An integer is mathematically optimal for representing a standalone year. It facilitates rapid chronological sorting (>, <) and relational indexing without the parsing overhead of a full DATE object.

Table: EXACT_DATE_

Elevated to a strong entity as the granular temporal reference for daily observations

- day_number, month_number, year_number (Composite PK) [INT]: Instead of relying on a monolithic DATE string which often causes import format mismatch errors, the date was structurally decomposed into its atomic integer values. INT ensures flawless mathematical validation and rapid indexing. year_number acts as a Foreign Key to the Year_ table.

Table: ENERGY_TYPE

Extracted to avoid hardcoding energy categories

- type_name (PK) [VARCHAR(255)]: Stores names such as 'solar' or 'nuclear'. VARCHAR(255) provides a safe, uniform margin.

Table: COMPANY

Single entity reused via two distinct FK roles in Power_Plant

- company_name (PK) [VARCHAR(255)]: Corporate names can be exceptionally long (e.g., consortiums). VARCHAR(255) prevents truncation.
- company_website [VARCHAR(255)]: URL strings. VARCHAR(255) is the standard safe allocation for URLs. NULLable.

Table: CITY

Weak entity dependent on COUNTRY. Composite PK handles homonyms

- city_name (PK) [VARCHAR(255)]: Sized uniformly with other descriptive text keys.
- country_name (PK, FK) [VARCHAR(255)]: Matches Country.country_name exactly to avoid implicit type casting during JOINs.
- city_rank [INT]: Numbeo ranking position. INT is mandatory for discrete ordinal values. NULLable.
- cost_of_living_index, rent_index, groceries_index, restaurant_price_index, local_purchasing_power_index, cost_of_living_plus_rent_index [FLOAT]: All socio-economic indices utilize FLOAT to strictly preserve the high decimal precision present in the Numbeo source CSV. NULLable.

Table: POWER_PLANT

Strong entity representing infrastructure

- power_plant_name (PK) [VARCHAR(255)]: Generous allocation for compound international facility names.
- power_plant_location, plant_type, power_plant_status [VARCHAR(100) to VARCHAR(255)]: Descriptive strings accommodating diverse lengths. NULLable.
- start_year [INT]: 4-digit operational year. INT is optimal for whole-number years.
- capacity_MW [INT]: Rated capacities in the source are whole numbers.
- annual_generation_GWh, lifetime_generation_GWh [FLOAT]: Energy generation metrics are highly precise continuous values (e.g., 2823.92). FLOAT prevents silent truncation. NULLable.
- country_name, operator_, owner_ (FKs) [VARCHAR(255)]: These keys map perfectly to their parent tables' VARCHAR(255) keys, enforcing relational integrity.

Table: GOVERNMENT_POLICY

Strong entity representing legislative frameworks.

- policy_id (PK) [INT]: Surrogate integer key. INT is standard for sequential, numeric identifiers.

- name_policy, policy_type, policy_status [VARCHAR(100) to VARCHAR(255)]: Descriptive strings mapped safely to avoid padding waste.
- budget_million_usd [INT]: Budgets are tracked in whole millions. INT abstracts smaller fractions to maintain a macroeconomic scale.
- target_reduction_percent [INT]: Represented as whole percentages in the dataset.
- start_policy_year [INT]: Maintains integer consistency with other temporal columns.
- country_name (FK) [VARCHAR(255)]: References Country. NOT NULL.

Table: REGISTERS

The most granular transactional entity (daily observations)

- city_name, country_name, day_number, month_number, year_number (Composite PK / FKs): Inherit their exact VARCHAR(255) and INT data types from the City and Exact_Date_ parent tables.
- avg_temperature, temperature, humidity, wind_speed, PM2_5, PM10, NO2, SO2, CO, O3 [FLOAT]: Every single meteorological and air quality measurement utilizes the FLOAT data type. Scientific atmospheric monitoring demands exact decimal preservation; assigning INT to these metrics would destroy the dataset's scientific validity. NULLable.

Table: YEARLY_STATS

Macro-economic facts independent of energy type

- country_name (PK, FK) [VARCHAR(255)], year_number (PK, FK) [INT]: Composite key mirroring parent types.
- population, gdp, energy_intensity_per_capita, energy_intensity_by_gdp [FLOAT]: GDP values reach trillions, and intensities contain complex fractions. FLOAT flawlessly accommodates the massive scale and necessary decimal precision. NULLable.

Table: YEARLY_EMITS

Stores sector-level CO2 emissions

- country_name (PK, FK) [VARCHAR(255)], year_number (PK, FK) [INT]: Keys mirroring parent types.
- co2_from_buildings, co2_from_industry, co2_from_land_use_change_and_forestry, co2_from_other_fuel_combustion, co2_from_transport, co2_from_manufacturing_and_construction, fugitive_co2_from_energy_production, co2_from_electricity_and_heat, co2_from_bunker_fuels [FLOAT]: All 9 sector columns utilize FLOAT. Crucially, FLOAT natively supports negative values, which is legally required to accurately store data for 'land_use_change_and_forestry' (acting as carbon sinks). NULLable.

Table: YEARLY_ENERGY_RECORDS

Ternary junction connecting Country, Year, and Energy Type

- country_name (PK, FK) [VARCHAR(255)], year_number (PK, FK) [INT], type_name (PK, FK) [VARCHAR(255)]: The three dimensions of the ternary key precisely matching their respective parent types.
- energy_consumption, energy_production, co2_emission [FLOAT]: Global energy consumption uses continuous, fractional values. FLOAT prevents the loss of crucial fractional data. NULLable.

General principles:

(1) VARCHAR(255) was heavily adopted for textual keys to act as a definitive shield against string truncation during massive CSV imports.

(2) FLOAT is exclusively used for all continuous numeric measurements to preserve scientific decimal precision directly from the source.

(3) INT is strictly used for discrete whole-number values such as IDs, numerical rankings, and atomic date components.

(4) Foreign key columns were engineered to share the exact length and data type as their referenced primary keys to prevent the engine from executing costly implicit conversions during table JOINs.

(5) NULL constraints were strategically left open for most fact metrics to accommodate historical data gaps without corrupting arithmetic functions with artificial zeros.

5 Physical model codification

The physical model script was developed to translate the theoretical Entity-Relationship (ER) diagram into a functional relational database schema. The creation process was strictly governed by relational algebra principles, focusing on data integrity, correct data type mapping, and dependency management.

To ensure the script executes without dependency violation errors, the creation order was meticulously engineered. The script is divided into four distinct topological phases: instantiating independent parent entities first, followed by weak entities, complex multi-dependent entities, and finally, the transactional associative tables.

Phase 1: Creation of Strong Entities (independent parent tables)

The script begins by constructing the foundation of the database: the strong entities that do not rely on any other tables for their existence.

- Country, Company, and Energy_Type: these tables were implemented using VARCHAR(255) for their primary keys (country_name, company_name, type_name) and descriptive attributes. This liberal character limit ensures that exceptionally long sovereign names or corporate titles are not truncated. The country_code attribute was constrained to VARCHAR(10) to optimize storage for standardized ISO codes.
- Year_: this one isolates the temporal dimension at the annual level. An INT data type was selected for the year_number primary key to facilitate rapid mathematical sorting and logical operations (e.g., comparing years using >= or <) without the overhead of parsing full date strings.

Phase 2: Creation of Weak Entities and Single-Dependency Tables

Once the foundation is established, the script generates entities that possess a direct hierarchical dependency (a single Foreign Key) on the Phase 1 tables.

- Exact_Date_: this table decomposes temporal data into its atomic components (day_number, month_number, year_number). A composite Primary Key utilizing all three INT columns was implemented to ensure chronological uniqueness. It establishes a FOREIGN KEY to the Year_ table, ensuring temporal consistency.
- Government_Policy: this entity maps environmental regulations to specific nations. A surrogate policy_id (INT) was chosen as the Primary Key for simplicity, while a FOREIGN KEY links it to the Country table. The financial attribute budget_million_usd utilizes an INT type, abstracting smaller monetary fractions to maintain macro-economic scale.
- City: a crucial design choice was made here. Because city names are not globally unique (e.g., "San Jose" exists in multiple countries), a Composite Primary Key was forged using both (city_name, country_name). The table imports country_name as a Foreign Key. Furthermore, to accommodate the high precision of socio-economic indices (like cost_of_living_index), the FLOAT data type was universally applied to these measurement attributes.

Phase 3: Creation of Entities with Multiple Dependencies

This phase translates entities that sit at the intersection of several strong entities, requiring multiple foreign keys to maintain referential integrity.

- Power_Plant: this table requires a complex web of relationships. It utilizes power_plant_name as its Primary Key. Crucially, it establishes three distinct Foreign Keys. One links to the Country table to denote geographical location. The other two (operator_ and owner_) are both Foreign Keys referencing the Company table. This is a sophisticated implementation of "role-playing" relationships, demonstrating that a single company can act simultaneously (or distinctly) as an owner and an operator of a facility. Energy generation metrics are stored as FLOAT to handle massive GWh outputs with decimal precision.

Phase 4: Creation of Transactional and Associative Tables (M:N Relationships)

The final segment of the script translates the "diamonds" from the ER diagram. These tables represent associative entities or transactional fact tables that record measurements over time and space. They rely heavily on massive composite primary keys formed entirely by foreign keys.

- Yearly_Stats and Yearly_Emits: these tables record national metrics aggregated by year. To guarantee that a country only has one statistical record per year, a composite Primary Key of (country_name, year_number) was enforced. Both columns simultaneously act as Foreign Keys to their respective parent tables. Environmental metrics and economic indicators (like gdp and various co2_from... categories) are stored as FLOAT to preserve scientific precision.
- Yearly_Energy_Records: this table resolves a ternary relationship. It tracks the production and consumption of a specific energy source, in a specific country, during a specific year. Therefore, its composite Primary Key requires three pillars: (country_name, year_number, type_name).
- Registers: this is the most granular transactional table in the schema, recording daily environmental and meteorological data per municipality. To ensure absolute uniqueness for every reading, the composite Primary Key mandates five columns: (city_name, country_name, day_number, month_number, year_number). It establishes complex Foreign Keys back to the composite PK of the City table and the composite PK of the Exact_Date_ table. Air quality measurements (PM2.5, NO2, SO2) utilize the FLOAT data type to accommodate strict scientific atmospheric monitoring standards.

6 Data import

The population of the physical database was executed utilizing the Oracle data import wizard (via drag-and-drop staging). This approach allowed for rapid bulk data ingestion. However, raw CSV (Comma-Separated Values) files are inherently untyped plain text. Consequently, the core methodology relied heavily on manual intervention during the import wizard's mapping phase.

When a CSV is dragged into the Oracle environment, the system utilizes a heuristic algorithm to scan the first 100 rows and guess the appropriate data types (e.g., defaulting to VARCHAR2(26) for short text or NUMBER for integers). This automated detection frequently conflicted with our meticulously designed physical model. Therefore, the primary methodology across all files consisted of:

1. Staging the CSV via the import wizard.
2. Intercepting the automated column mapping.
3. Manually adjusting the data types to match our script (e.g., expanding VARCHAR2 limits, correcting decimal separators for FLOAT metrics, and defining explicit date format masks).
4. Mapping the CSV columns sequentially to our pre-created relational tables, ensuring parent tables (strong entities) were fully populated before child tables (weak entities) to prevent Foreign Key constraint violations.

6.1 nuclear_power_dataset.csv

Data filing and pre-processing

The first major dataset integrated into the database was nuclear_power_dataset.csv, which served as the foundational source for our infrastructure-related tables: Power_Plant, Company, and the initial instantiation of the Country table. An inspection of the raw CSV revealed a highly denormalized structure, containing specific string formats for corporate entities (e.g., "Smith, Thompson and Alvarez") and high-precision scientific floats for lifetime generation metrics.

Import execution and table mapping

Following our strictly Entity-Relationship model, this file could not be imported directly into the Power_Plant table in a single pass. A multi-phase extraction methodology via the Oracle Data Import Wizard was strictly required. First, the CSV was staged to extract distinct sovereign states to populate the Country parent table, ensuring no duplicate entries were generated. Subsequently, the same file was processed again, mapping only the unique values from the 'Owner' and 'Operator' columns to populate the Company parent table. Once these strong entities were established, the final drag-and-drop import was executed to populate the central Power_Plant table, utilizing the newly created constraints to enforce referential integrity.

Data type and integrity resolution

During the mapping phase, Oracle's heuristic detection mechanism triggered several errors that required manual intervention:

- String truncation risks: Oracle instinctively assigned VARCHAR2(50) to the Power_Plant_Name, Owner, and Operator attributes based on the first 100 rows. Knowing that international and facility names can be exceptionally long, this was manually overridden and expanded to VARCHAR2(255) to prevent fatal ORA-12899.

- Numeric precision and floating points: attributes such as Capacity_MW and Annual_Generation_GWh were initially interpreted by Oracle as standard integers (INT). Because energy generation operates on precise decimals (e.g., 1604.345...), we manually redefined these target columns as FLOAT to prevent silent truncation of critical scientific data.

6.2 co-emissions-by-sector.csv

Data filing and pre-processing.

The co-emissions-by-sector.csv file was imported to populate the Yearly_Emits transactional table. This dataset categorizes carbon footprints across highly specific industrial and social sectors per country, per year. The primary challenge identified during profiling was the extreme length of the CSV headers (e.g., "Carbon dioxide emissions from buildings") and the presence of missing data points for specific nations in earlier decades.

Import execution and table mapping.

The import methodology required strict mapping to our physical schema. We mapped the raw CSV columns to their aliases (e.g., mapping the raw "Carbon dioxide emissions from electricity and heat" column directly to our co2_from_electricity_and_heat physical attribute).

Data type and typological corrections

- Handling regions: the raw CSV contained pre-aggregated macroeconomic regions (e.g., "High-income countries"). Because our Yearly_Emits table enforces a strict Foreign Key to the Country table, importing these non-sovereign entities would violate relational integrity. The import validation was leveraged to automatically reject and drop these orphaned records during the load process, ensuring the table remained perfectly normalized.
- Null value: the CSV contained empty delimiters for countries lacking historical sector data. We explicitly configured the wizard to interpret these blanks as native SQL NULL values rather than accepting them into artificial zeroes (0), thereby protecting the statistical integrity of future SUM() and AVG() aggregations.

6.3 energy.csv

Data filing and pre-processing

The energy.csv dataset was crucial for populating the Yearly_Stats and Yearly_Energy_Records tables. It acts as a massive temporal matrix containing macro-economic indicators (GDP, Population) alongside granular energy production and consumption metrics filtered by energy type (e.g., "coal", "natural_gas").

Import execution and data type

- Temporal type mapping: Oracle's automated detection attempted to cast the Year column as a standard DATE datatype, appending artificial day and month prefixes. To adhere to our physical model, this was manually overridden in the mapping interface, forcing the engine to parse the column strictly as an INT to match our Year_ table's Primary Key.
- Scientific notation resolution: several global metrics within the CSV (such as total global population or GDP) were formatted using scientific notation due to their massive scale. Advanced formatting properties within the Oracle tool were applied to ensure these exponential values were correctly parsed and mathematically translated into standard FLOAT datatypes without losing numerical precision.

6.4 Cost_of_living_index.csv

Data Filing and pre-processing

This file was dedicated to populating our socio-economic City table. During the initial profiling phase, a significant structural anomaly was detected: the CSV contained a single City column where both the municipality and the sovereign state were concatenated together within quotation marks (e.g., "Cairo, Egypt" or "Hamilton, Bermuda").

Data and transformation methodology

Because our physical model explicitly requires a composite Primary Key consisting of separate city_name and country_name attributes (with the latter acting as a Foreign Key), a direct import was mathematically impossible. To resolve this, a staging methodology was done. The data was first imported into a temporary staging table where Oracle's substring functions (e.g., utilizing the comma as a delimiter) were applied to split the raw column into two distinct relational columns. Once the data was successfully decoupled, an INSERT INTO ... SELECT query was utilized to move the clean, separated data into the final City table. Furthermore, international decimal separators (periods) in the index columns were explicitly declared in the wizard to ensure the FLOAT indices were parsed correctly.

6.5 global_air_quality_data10000.csv & city_temperature.csv

Data filing and pre-processing

These two massive files were combined to populate our most granular transactional entity: the Registers table, as well as instantiating the temporal Exact_Date_ table. The global_air_quality_data_10000.csv provided PM2.5, NO2, and O3 metrics, while city_temperature.csv provided historical climate averages.

Import execution and chronological parsing (the "Date" problem)

The challenge for this import phase was chronological decomposition. Our ER model dictates that temporal registers must be stored utilizing an atomic composite key: day_number, month_number, and year_number. However, the global_air_quality_data_10000.csv provided dates in a monolithic string format (e.g., YYYY-MM-DD).

- Resolution: during the import phase, advanced extraction methodologies were required. Rather than fundamentally altering our normalized database structure, the date string was parsed during the staging phase, decomposing it into three distinct integer columns.
- Composite constraint verification: because the Registers table utilizes a highly complex five-column composite primary key (city_name, country_name, day_number, month_number, year_number), the import wizard was rigorously configured. This ensured that every single daily register was explicitly mapped to a pre-existing City and Exact_Date record. Any orphaned climate readings belonging to unregistered cities were systematically rejected by the engine, maintaining absolute domain integrity.

6.6 Government_Policy.csv

Data filing and pre-processing

The final dataset, Government_Policy.csv, cataloged environmental frameworks. It was utilized to populate the Government_Policy table. The dataset featured a highly structured format, including policy statuses, financial budgets, and target reduction percentages.

Import execution and table mapping

The import process for this file was relatively streamlined due to its pre-normalized structure. The policy_id was mapped directly to our Primary Key. The primary manual intervention involved ensuring the budget_million_usd and target_reduction_percent columns were correctly parsed as INT types, overriding Oracle's tendency to classify them as generic NUMBER types. Additionally, strict referential integrity was enforced during the load: the wizard verified that every country_name referenced in the policy CSV explicitly existed within our Country parent table, successfully concluding the database population phase without structural violations.

7 Query implementation

7.1 (Which power plants have been operational for more than 30 years?)

```sql
SELECT
    power_plant_name,
    country_name,
    start_year,
    (2026 - start_year) AS "Years Operational"
FROM
    Power_Plant
WHERE
    power_plant_status = 'Operational'
    AND (2026 - start_year) > 30
ORDER BY
    "Years Operational" DESC;
```

Resultado de la Consulta | Salida de Script | Salida de DBMS | Explicación del plan | Rastreo automático | Hist

Descargar | Tiempo de ejecución: 0,01 segundos

	POWER_PLANT_NA	COUNTRY_NAME	START_YEAR	YEARS OPERATIONAL
132	Brown, Moss and ...	Antigua and Barb...	1992	34
133	Vaughan-Johnson	United Arab Emira...	1992	34
134	Stephens Ltd	Lao People's Dem...	1992	34
135	Lawrence PLC	Lesotho	1992	34
136	Espinoza PLC	Saint Vincent and ...	1993	33
137	Ayala-Tucker	San Marino	1993	33
138	Barnes-Figueroa	Tunisia	1993	33
139	Howe-Spencer	Saint Pierre and M...	1993	33
140	Casey, Patterson a...	Japan	1993	33
141	Cook, Huerta and ...	Gibraltar	1993	33
142	Martinez PLC	Marshall Islands	1993	33
143	Carr, Chapman an...	Aruba	1993	33
144	Graham and Sons	Belgium	1994	32
145	Mitchell, Villanuev...	Cyprus	1994	32
146	Lewis LLC	China	1995	31
147	Jones and Sons	Georgia	1995	31
148	Harris LLC	Wallis and Futuna	1995	31
149	Nicholson Ltd	American Samoa	1995	31

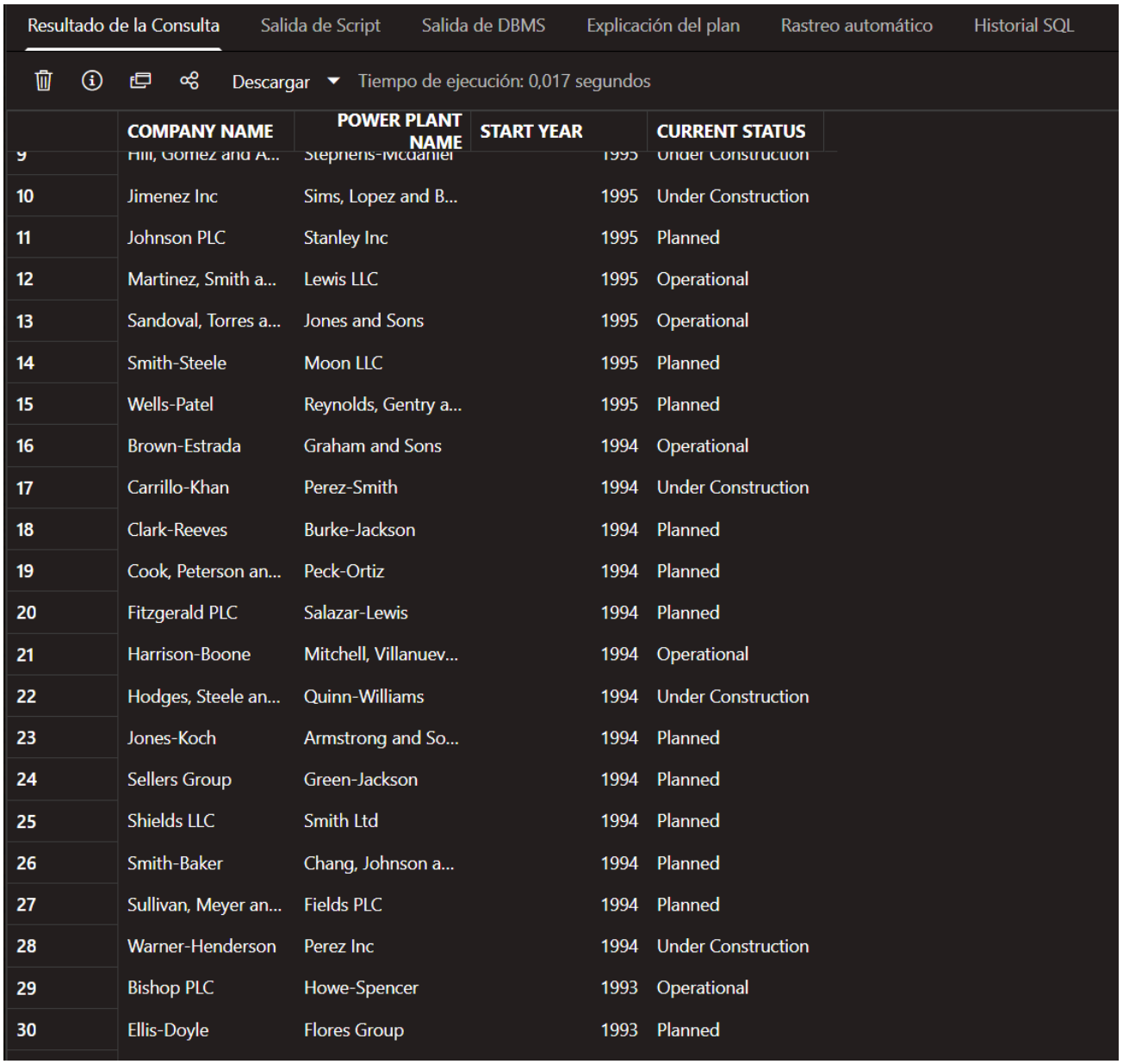
7.1.1 Explanation

This query is designed to identify active nuclear power plants that have been operational for more than 30 years by dynamically calculating their age. To achieve this, we utilize the number 2026 (current year) which is then subtracted from the start_year attribute to compute the exact operational age without relying on hardcoded dates. A WHERE clause is implemented to filter the results using two strict conditions: ensuring the power_plant_status is explicitly 'Operational' and that the calculated age strictly exceeds 30. Furthermore, the ORDER BY ... DESC tool is applied to sort the final dataset in descending order, guaranteeing that the oldest and most historically significant plants appear at the top.

7.1.2 Validation

Rather than mathematically calculating the age via subtraction, this control query isolates the dataset using the absolute temporal cutoff of 1995 (which strictly satisfies the >30 years requirement relative to the 2026 baseline). By retrieving all corporate owners and facilities established in or before 1995 alongside their respective power_plant_status, we can explicitly observe the coexistence of both active and decommissioned plants

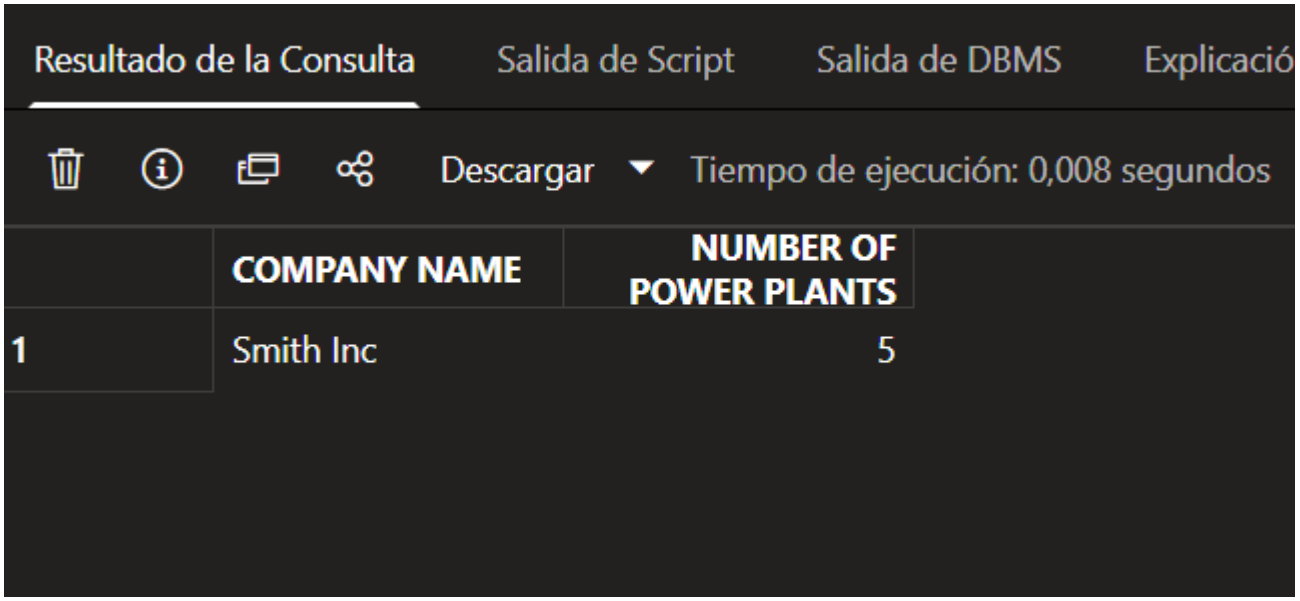
```
SELECT
    owner_ AS "Company Name",
    power_plant_name AS "Power Plant Name",
    start_year AS "Start Year",
    power_plant_status AS "Current Status"
FROM
    Power_Plant
WHERE
    start_year <= 1995
ORDER BY
    start_year DESC, "Company Name" ASC;
```



	COMPANY NAME	POWER PLANT NAME	START YEAR	CURRENT STATUS
9	Hill, Gomez and A...	Stephens-Mcdaniel	1995	Under Construction
10	Jimenez Inc	Sims, Lopez and B...	1995	Under Construction
11	Johnson PLC	Stanley Inc	1995	Planned
12	Martinez, Smith a...	Lewis LLC	1995	Operational
13	Sandoval, Torres a...	Jones and Sons	1995	Operational
14	Smith-Steele	Moon LLC	1995	Planned
15	Wells-Patel	Reynolds, Gentry a...	1995	Planned
16	Brown-Estrada	Graham and Sons	1994	Operational
17	Carrillo-Khan	Perez-Smith	1994	Under Construction
18	Clark-Reeves	Burke-Jackson	1994	Planned
19	Cook, Peterson an...	Peck-Ortiz	1994	Planned
20	Fitzgerald PLC	Salazar-Lewis	1994	Planned
21	Harrison-Boone	Mitchell, Villanuev...	1994	Operational
22	Hodges, Steele an...	Quinn-Williams	1994	Under Construction
23	Jones-Koch	Armstrong and So...	1994	Planned
24	Sellers Group	Green-Jackson	1994	Planned
25	Shields LLC	Smith Ltd	1994	Planned
26	Smith-Baker	Chang, Johnson a...	1994	Planned
27	Sullivan, Meyer an...	Fields PLC	1994	Planned
28	Warner-Henderson	Perez Inc	1994	Under Construction
29	Bishop PLC	Howe-Spencer	1993	Operational
30	Ellis-Doyle	Flores Group	1993	Planned

7.2 (Which companies own more than 3 power plants?)

```sql
SELECT
    owner_ AS "Company Name",
    COUNT(power_plant_name) AS "Number of Power Plants"
FROM
    Power_Plant
WHERE
    owner_ IS NOT NULL
GROUP BY
    owner_
HAVING
    COUNT(power_plant_name) > 3
ORDER BY
    "Number of Power Plants" DESC;
```



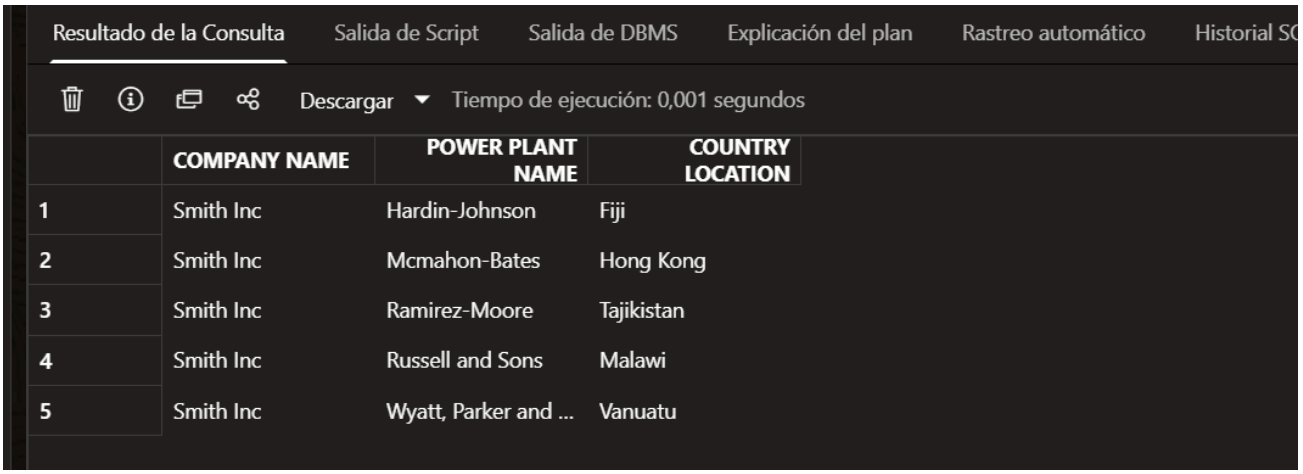
7.2.1 Explanation

This query is designed to identify corporate entities that own more than three nuclear power plants by aggregating data from the Power_Plant table. We utilize the GROUP BY clause on the owner_ attribute to cluster the individual plant records, ensuring that each company is evaluated as a single aggregated entity. The COUNT() function is applied to calculate the total number of distinct facilities associated with each owner. Crucially, the HAVING clause is implemented instead of a standard WHERE clause, as it is strictly required by SQL to filter results based on aggregated metrics (in this case, COUNT > 3). Finally, the dataset is ordered in descending fashion to highlight the corporations with the largest nuclear portfolios at the top.

7.2.2 Validation

To prove the COUNT() function is accurate, we select one specific corporation from our main results (for example, if the main query states that 'Smith Inc' owns 50 plants) and extract the raw, unaggregated rows.

```
SELECT
    owner_ AS "Company Name",
    power_plant_name AS "Power Plant Name",
    country_name AS "Country Location"
FROM
    Power_Plant
WHERE
    owner_ = 'Smith Inc'
ORDER BY
    power_plant_name ASC;
```



	COMPANY NAME	POWER PLANT NAME	COUNTRY LOCATION
1	Smith Inc	Hardin-Johnson	Fiji
2	Smith Inc	Mcmahon-Bates	Hong Kong
3	Smith Inc	Ramirez-Moore	Tajikistan
4	Smith Inc	Russell and Sons	Malawi
5	Smith Inc	Wyatt, Parker and ...	Vanuatu

7.3 (Which countries produce more energy than they consume in 2015?

```sql
SELECT c.country_name

FROM Country c

JOIN yearly_energy_records yer ON c.country_name = yer.country_name

WHERE yer.year_number = 2015

GROUP BY c.country_name

HAVING SUM(yer.energy_production) > SUM(yer.energy_consumption);
```

	COUNTRY_NAME	TOTAL_PRODUCT
1	Suriname	0.08282154839...
2	Bhutan	0.146946717330...
3	Congo-Kinshasa	0.250 0.1469467173306617773
4	Yemen	0.407519681769...
5	Cameroon	0.55707464255...
6	Chad	0.5602951914
7	South Sudan	0.566528717384
8	Ghana	0.60468675591...
9	Timor-Leste	0.748853305
10	Gabon	0.96206554355...
11	Papua New Guinea	0.979454236812
12	Paraguay	1.041118668
13	Mozambique	1.127993038526...
14	Congo-Brazzaville	1.159667524632...

7.3.1 Explanation

This query identifies sovereign nations whose aggregate energy production surpassed their total energy consumption during the fiscal year 2015. Since a single country may have multiple records across different energy types within the yearly_energy_records table, a GROUP BY on country_name is applied to consolidate all energy type contributions per nation. The HAVING clause then performs the comparative filtering post-aggregation, ensuring only countries with a net energy surplus are returned. Results are ordered by total production in ascending order to facilitate analysis from the lowest to the highest producers.

7.3.2 Validation

To verify correctness, the validation query extends the main query by exposing the raw aggregated values total_production, total_consumption, and the calculated surplus column. Every row returned must display a strictly positive surplus value. Additionally, reversing the HAVING condition to SUM(energy_production) < SUM(energy_consumption) should return a disjoint set of countries, confirming the two result sets are mutually exclusive and collectively exhaustive.

```
SELECT

    c.country_name,

    SUM(yer.energy_production) AS total_production,

    SUM(yer.energy_consumption) AS total_consumption,

    SUM(yer.energy_production) - SUM(yer.energy_consumption) AS surplus

FROM Country c

JOIN yearly_energy_records yer ON c.country_name = yer.country_name

WHERE yer.year_number = 2015

GROUP BY c.country_name

HAVING SUM(yer.energy_production) > SUM(yer.energy_consumption)

ORDER BY surplus DESC;
```

	COUNTRY_NAME	TOTAL_PRODUCTIC	TOTAL_CONSUMPT	SURPLUS
1	Russia	115.33803584023...	61.898838959199...	53.439196881034...
2	Saudi Arabia	57.456764108236...	21.299664814767...	36.157099293469...
3	Australia	31.304398953626...	11.555558025403...	19.748840928222...
4	Qatar	19.56474964144614	3.984215183150696	15.580534458295...
5	Norway	19.062664615930...	3.8505453509598...	15.212119264970...
6	Indonesia	28.590556237831...	13.394256581529...	15.196299656301...
7	Iraq	17.33965583504369	2.86961396922	14.47004186582369
8	Canada	41.828935282091...	29.411371043581...	12.417564238510...
9	Kuwait	13.909589549919...	3.3522515925726...	10.557337957346...
10	United Arab Emira...	19.453777487211...	9.4043996106750...	10.049377876536...
11	Nigeria	12.909679482965...	3.1339036431268...	9.7757758398390...
12	Venezuela	14.832053109878...	6.1201860872328...	8.7027661216460...

7.4(Which energy type has the highest average CO_2 emission across all countries?)

```
SELECT yer.type_name

FROM yearly_energy_records yer

WHERE yer.type_name != 'all_energy_types'

GROUP BY yer.type_name

ORDER BY AVG(yer.co2_emission) DESC

FETCH FIRST 1 ROWS ONLY;
```

	TYPE_NAME
1	coal

7.4.1 Explanation

This query identifies the energy type that recorded the highest average CO_2 emission across all countries and years within the database. The yearly_energy_records table is grouped by type_name, and AVG(co2_emission) is computed per energy type. The aggregate row 'all_energy_types' is explicitly excluded from the analysis via the WHERE clause, as it represents a macro-level summation across all energy sources rather than an individual energy type, and its inclusion would skew the results. The ORDER BY AVG(co2_emission) DESC combined with FETCH FIRST 1 ROWS ONLY ensures that only the single highest-ranking energy type is returned.

7.4.2 Validation

The validation query removes the FETCH FIRST 1 ROWS ONLY restriction and exposes the full ranked list of all energy types alongside their average CO_2 emission values. The energy type returned by the main query must appear at the top of this ranked list. The 'all_energy_types' aggregate row is equally excluded from the validation to maintain consistency. Any other ordering would indicate an error in the aggregation or sorting logic.

```
SELECT
    yer.type_name,
    AVG(yer.co2_emission) AS avg_co2
FROM yearly_energy_records yer
WHERE yer.type_name != 'all_energy_types'
GROUP BY yer.type_name
ORDER BY avg_co2 DESC;
```

	TYPE_NAME	AVG_CO2
1	coal	103.27483395291...
2	petroleum_n_othe...	96.165092058182...
3	natural_gas	46.626336226581...
4	nuclear	0
5	renewables_n_other	0

7.5 (Which countries have increased their energy consumption every year between 2010 and 2020?)

```
SELECT actual.country_name AS "Country Name"

FROM Yearly_Energy_Records actual

LEFT JOIN Yearly_Energy_Records previo ON actual.country_name = previo.country_name

  AND actual.type_name = previo.type_name

  AND actual.year_number = previo.year_number + 1

WHERE actual.type_name = 'all_energy_types' AND actual.year_number BETWEEN 2010 AND 2019

GROUP BY actual.country_name

HAVING COUNT(actual.year_number) = 10 AND SUM(

       CASE

           WHEN previo.energy_consumption IS NOT NULL AND actual.energy_consumption <=
previo.energy_consumption THEN 1

           ELSE 0

       END

   ) = 0

ORDER BY "Country Name" ASC;
```



	COUNTRY NAME
1	Bangladesh
2	Cambodia
3	Costa Rica
4	Ethiopia
5	Former Czechoslo...
6	Former Serbia and...
7	Former U.S.S.R.
8	Former Yugoslavia
9	Germany, East
10	Germany, West
11	Guatemala
12	Hawaiian Trade Zo...
13	India
14	Laos
15	Mali
16	Mauritius
17	Peru
18	Rwanda
19	World

7.5.1 Explanation

This query identifies sovereign nations whose total energy consumption grew strictly year-over-year across every consecutive year from 2010 to 2019. To achieve this without relying on nested subqueries or analytical window functions, the analysis utilizes a self-join technique. The yearly energy records table is instantiated twice within the same scope, aliased as 'actual' and 'previo'. By joining these two instances on the country name, energy type, and specifically matching the current year to the previous year plus one, the relational engine aligns each annual record directly alongside its historical predecessor in a single flat structure. The query is then aggregated using the GROUP BY clause, where the HAVING clause applies a strict two-fold mathematical condition. First, it verifies that the country possesses exactly ten valid annual records for the decade. Second, it evaluates a conditional sum that flags any instance where the current energy consumption fails to exceed the previous year's metric; ensuring this sum evaluates exactly to zero guarantees a flawless and continuous upward consumption trajectory.

7.5.2 Validation

To substantiate the results of the primary query, the validation query exposes the raw year-by-year consumption values alongside the previous year's baseline figure for a sample of the successfully returned countries, such as Bangladesh, India, or Peru. Utilizing the exact same self-join architecture, this query isolates these specific nations to allow for manual verification of the continuous upward trend. Every single row returned must display a current energy consumption value strictly greater than the previous consumption value attached to it. If any year displayed a decrease or remained stagnant, the main query's conditional sum logic would have successfully flagged the mathematical anomaly and excluded that sovereign entity from the final dataset, thereby proving the structural integrity of the relational filter.

```sql
SELECT
    actual.country_name AS "Country",
    actual.year_number AS "Year",
    actual.energy_consumption AS "Current Consumption",
    previo.energy_consumption AS "Previous Consumption"
FROM Yearly_Energy_Records actual
LEFT JOIN Yearly_Energy_Records previo
  ON actual.country_name = previo.country_name
  AND actual.type_name = previo.type_name
  AND actual.year_number = previo.year_number + 1
WHERE actual.type_name = 'all_energy_types'
  AND actual.year_number BETWEEN 2010 AND 2019
  AND actual.country_name IN ('Bangladesh', 'India', 'Peru')
ORDER BY actual.country_name ASC, actual.year_number ASC;
```

Resultado de la Consulta | Salida de Script | Salida de DBMS | Explicación del plan | Rastreo

Descargar | Tiempo de ejecución: 0,016 segundos

	COUNTRY	YEAR	CURRENT CONSUMPTION	PREVIOUS CONSUMPTION
9	Bangladesh	2018	1.5503656410674...	1.459108195914739
10	Bangladesh	2019	1.6166327357480...	1.5503656410674...
11	India	2010	22.201383939498...	20.798171377599...
12	India	2011	23.113729753852...	22.201383939498...
13	India	2012	24.270109803581...	23.113729753852...
14	India	2013	24.86770283637093	24.270109803581...
15	India	2014	26.636202250449...	24.86770283637093
16	India	2015	27.180582161810...	26.636202250449...
17	India	2016	28.09995455336339	27.180582161810...
18	India	2017	29.463450235998...	28.09995455336339
19	India	2018	31.198004677089...	29.463450235998...
20	India	2019	31.782739499292...	31.198004677089...
21	Peru	2010	0.8644234130371...	0.7705799312773...
22	Peru	2011	0.9048489508713...	0.8644234130371...
23	Peru	2012	0.9597099019199...	0.9048489508713...
24	Peru	2013	0.9874629225056...	0.9597099019199...
25	Peru	2014	1.0062818042237...	0.9874629225056...
26	Peru	2015	1.0466049422164...	1.0062818042237...
27	Peru	2016	1.0846614653746...	1.0466049422164...
28	Peru	2017	1.0937353126498...	1.0846614653746...
29	Peru	2018	1.1359035392231...	1.0937353126498...
30	Peru	2019	1.1855060965141...	1.1359035392231...

7.6 (Which countries have a GDP above the world average GDP in 2015?)

```sql
SELECT ys.country_name
FROM yearly_stats ys
WHERE ys.year_number = 2015
AND ys.gdp > (
    SELECT AVG(ys2.gdp)
    FROM yearly_stats ys2
    WHERE ys2.year_number = 2015
    AND ys2.country_name != 'World'
)
AND ys.country_name != 'World'
ORDER BY ys.gdp DESC;
```

	COUNTRY_NAME
1	United States
2	China
3	India
4	Japan
5	Germany
6	Russia
7	Brazil
8	United Kingdom
9	France
10	Indonesia
11	Italy
12	Mexico
13	Turkey

7.6.1 Explanation

This query identifies countries whose GDP in 2015 exceeded the global average GDP for that same year. The analysis is restricted to 'all_energy_types' to avoid duplicate GDP values across energy type rows. The 'World' aggregate row is explicitly excluded from both the main query and the scalar subquery to prevent macro-level distortion. The subquery dynamically computes the average GDP threshold, making the query robust to data changes.

7.6.2 Validation

The validation query exposes each country's GDP alongside the computed world average threshold. Every country returned must have a gdp value strictly greater than world_avg_gdp. Countries appearing below the threshold would indicate an error in the filtering logic.

```
SELECT

    ys.country_name,

    ys.gdp,

    (SELECT AVG(ys2.gdp)

     FROM yearly_stats ys2

     WHERE ys2.year_number = 2015

     AND ys2.country_name != 'World') AS world_avg_gdp

FROM yearly_stats ys

WHERE ys.year_number = 2015

AND ys.country_name != 'World'

AND ys.gdp IS NOT NULL

ORDER BY ys.gdp DESC;
```

	COUNTRY_NAME	GDP	WORLD_AVG_GDP
1	United States	18206.03	551.40835338916...
2	China	17895.85	551.40835338916...
3	India	7163.88	551.40835338916...
4	Japan	5200.87	551.40835338916...
5	Germany	3890.031	551.40835338916...
6	Russia	3523.119	551.40835338916...
7	Brazil	3013.558	551.40835338916...
8	United Kingdom	2772.521	551.40835338916...
9	France	2719.151	551.40835338916...
10	Indonesia	2647.955	551.40835338916...
11	Italy	2241.422	551.40835338916...
12	Mexico	2230.743	551.40835338916...
13	Turkey	2025.977	551.40835338916...
14	South Korea	1933.636	551.40835338916...
15	Saudi Arabia	1708.35	551.40835338916...

7.7 (Which countries have more than 5 power plants?)

```
SELECT pp.country_name

FROM Power_Plant pp

GROUP BY pp.country_name

HAVING COUNT(pp.power_plant_name) > 5

ORDER BY COUNT(pp.power_plant_name) DESC;
```

	COUNTRY_NAME
1	Palestinian Territory
2	Albania
3	British Virgin Islands
4	Svalbard & Jan M...
5	Myanmar
6	Italy
7	Pakistan
8	North Macedonia
9	Cook Islands
10	Belarus
11	Luxembourg
12	Mali
13	Ghana
14	Tanzania
15	Netherlands
16	Congo

7.7.1 Explanation

This query identifies countries that host more than 5 power plants in the database. The Power_Plant table is grouped by country_name and COUNT is applied on power_plant_name to tally the number of plants per country. The HAVING clause filters out any country with 5 or fewer plants.

7.7.2 Validation

The validation query exposes the exact plant count per country. Every row must show a total_plants value strictly greater than 5. The results are ordered descending to easily verify the highest-count countries at the top.

```
SELECT
    pp.country_name,
    COUNT(pp.power_plant_name) AS total_plants
FROM Power_Plant pp
GROUP BY pp.country_name
HAVING COUNT(pp.power_plant_name) > 5
ORDER BY total_plants DESC;
```

	COUNTRY_NAME	TOTAL_PLANTS
1	Palestinian Territory	12
2	Albania	10
3	British Virgin Islands	10
4	Svalbard & Jan M...	10
5	Myanmar	10
6	Italy	9
7	Pakistan	9
8	North Macedonia	8
9	Cook Islands	8
10	Belarus	8
11	Luxembourg	8
12	Mali	8
13	Ghana	8
14	Tanzania	7
15	Netherlands	7

7.8 (What is the average capacity (MW) of power plants grouped by type)

```
SELECT pp.plant_type,

    AVG(pp.capacity_MW) AS avg_capacity

FROM Power_Plant pp

GROUP BY pp.plant_type

ORDER BY avg_capacity DESC;
```

	PLANT_TYPE	AVG_CAPACITY
1	Fusion	1058.0421052631...
2	Nuclear	1050.5181451612...

7.8.1 Explanation

This query computes the average installed capacity in megawatts for each power plant type recorded in the database. The Power_Plant table is grouped by plant_type and AVG(capacity_MW) is applied per group. Results are ordered descending to immediately identify the most capacity-intensive plant types.

7.8.2 Validation

The validation query extends the analysis by also exposing the total plant count, minimum and maximum capacity per type. This allows verification that the average falls within the expected range between min_capacity and max_capacity for every plant type.

```
SELECT

    pp.plant_type,

    COUNT(pp.power_plant_name) AS total_plants,

    MIN(pp.capacity_MW) AS min_capacity,

    MAX(pp.capacity_MW) AS max_capacity,

    AVG(pp.capacity_MW) AS avg_capacity

FROM Power_Plant pp

GROUP BY pp.plant_type

ORDER BY avg_capacity DESC;
```

	PLANT_TYPE	TOTAL_PLANTS	MIN_CAPACITY	MAX_CAPACITY	AVG_CAPACITY
1	Fusion	475	104	1986	1058.0421052631...
2	Nuclear	496	104	2000	1050.5181451612...

7.9 (Which power plants have an annual generation above the average generation of all plants?)

```
SELECT pp.POWER_PLANT_NAME, AVG(pp.ANNUAL_GENERATION_GWH) AS AnnualGeneration
FROM POWER_PLANT pp
GROUP BY pp.POWER_PLANT_NAME
HAVING AnnualGeneration > (
    SELECT AVG(pp2.ANNUAL_GENERATION_GWH)
    FROM POWER_PLANT pp2
)
ORDER BY AnnualGeneration ASC
```

	POWER_PLANT_NA	ANNUALGENERATI
1	Good-Conrad	5148.912301596777
2	Cobb and Sons	5157.648091226413
3	Hendrix-Figueroa	5158.588001378377
4	Ruiz, Green and N...	5160.925372515421
5	Hardin-Johnson	5163.281236949458
6	Wise, Rodriguez a...	5175.552130321789
7	Smith-Graham	5179.922314033786
8	Solis, Washington ...	5213.987225018262
9	Pineda, Cumming...	5215.11591376395
10	Lewis-Rivas	5227.731902500609
11	Barrett Inc	5232.099264734282
12	Smith, Meadows a...	5233.560059263734
13	Downs-Schultz	5234.5870555356...
14	Sweeney and Sons	5247.53676775577
15	Phillips-Luna	5262.15688764883
16	Barry-Davis	5262.584872439277

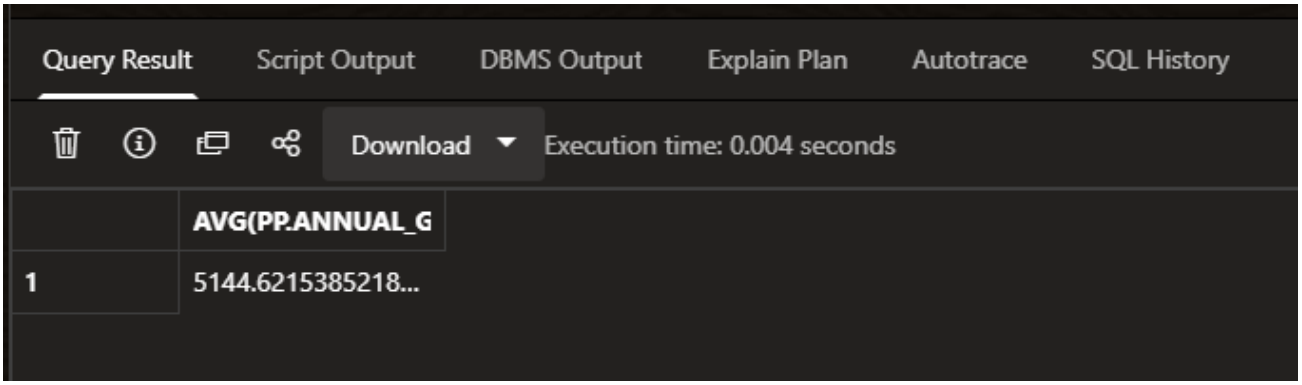
7.9.1 Explanation

This query identifies power plants whose average annual generation is higher than the overall average annual generation of all power plants. The query groups the data by power_plant_name and calculates the average annual_generation_gwh for each plant. The HAVING clause is used because the condition depends on an aggregate value. The subquery calculates the global average annual generation, and only plants above that value are returned.

7.9.2 Validation

This validation query returns the global average used as the threshold in the main query. Every power plant returned by the main query should have an annual generation greater than this value.

```
SELECT AVG(pp.ANNUAL_GENERATION_GWH)
FROM POWER_PLANT pp
```



7.10 (Which cities are both economically expensive (Cost of Living Index > 80) and experience significant particulate pollution (Average PM2.5 > 30?)

```
SELECT city_name AS "City"

FROM City

WHERE cost_of_living_index > 80

INTERSECT

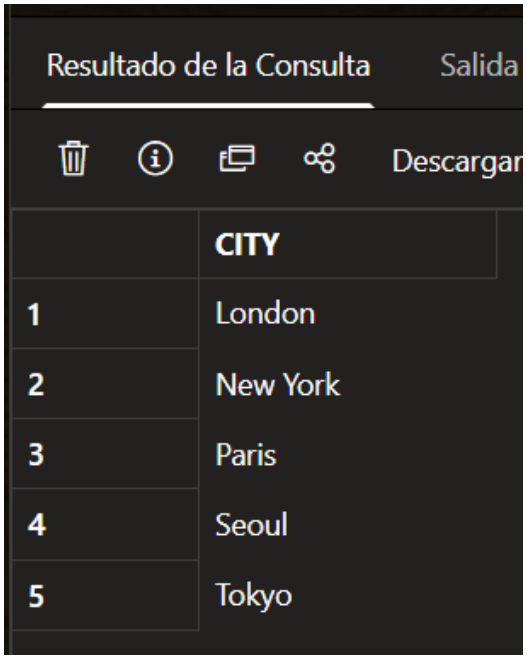
SELECT city_name AS "City"

FROM Registers

GROUP BY city_name

HAVING AVG(pm2_5) > 30

ORDER BY "City" ASC;
```



	CITY
1	London
2	New York
3	Paris
4	Seoul
5	Tokyo

7.10.1 Explanation

This query utilizes the relational INTERSECT set operator to identify municipalities that suffer from compounding negative conditions: high economic strain and poor air quality. Instead of executing a traditional inner join, the query independently extracts two distinct sets of data. The first sub-query isolates cities from the static economic dimension that possess a cost of living index exceeding a baseline of 80. The second sub-query aggregates millions of daily transactional records from the registers table to isolate cities where the historical average of fine particulate matter (PM2.5) exceeds 30 micrograms per cubic meter. The mathematical intersection of these two autonomous sets elegantly filters the output, returning only the exact city names that simultaneously exist in both highly restrictive datasets, thereby highlighting urban centers with severe cost and health risks.

7.10.2 Validation

To substantiate the results generated by the intersection operator, the validation query abandons the set-theory approach and instead relies on a strict explicit inner join. By isolating a specific sample of the cities returned by the main query (such as London, Paris, or New York), this validation visually exposes both the raw economic index and the mathematically aggregated pollution metric side-by-side. For the intersection to be mathematically valid, every single row returned by this validation query must explicitly display a cost of living index greater than 80 and a calculated average PM2.5 value strictly greater than 30. If any city failed to meet both explicit thresholds simultaneously, it would not have survived the relational intersection, proving the absolute structural integrity of the primary query's logic.

```
SELECT

    c.city_name AS "City",

    c.cost_of_living_index AS "Cost of Living Index",

    ROUND(AVG(r.pm2_5), 2) AS "Average PM2.5"

FROM City c

INNER JOIN Registers r

  ON c.city_name = r.city_name

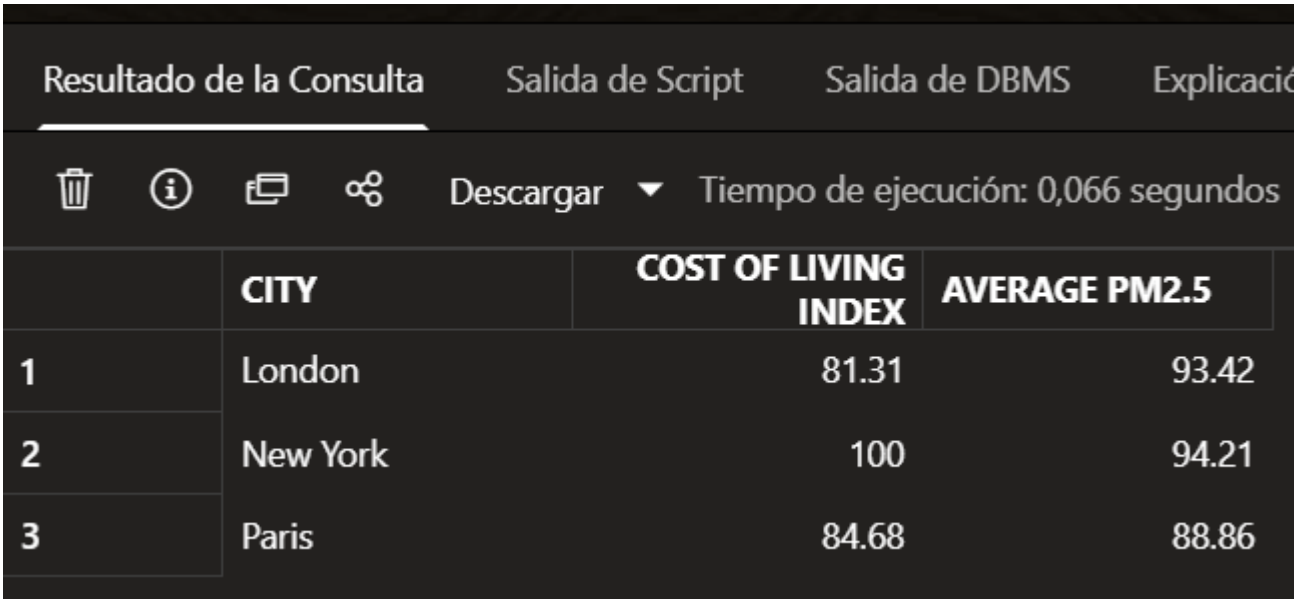
  AND c.country_name = r.country_name

WHERE c.city_name IN ('London', 'Paris', 'New York')

GROUP BY c.city_name, c.cost_of_living_index

HAVING c.cost_of_living_index > 80 AND AVG(r.pm2_5) > 30

ORDER BY "City" ASC;
```



	CITY	COST OF LIVING INDEX	AVERAGE PM2.5
1	London	81.31	93.42
2	New York	100	94.21
3	Paris	84.68	88.86

7.11 (Which countries have an energy intensity per capita above the world average in 2015?)

```sql
SELECT
    country_name AS "Country",
    energy_intensity_per_capita AS "Energy Intensity per Capita (2015)"
FROM
    Yearly_Stats
WHERE
    year_number = 2015
    AND country_name != 'World'
    AND energy_intensity_per_capita > (
        SELECT AVG(energy_intensity_per_capita)
        FROM Yearly_Stats
        WHERE year_number = 2015 AND country_name != 'World'
    )
ORDER BY
    "Energy Intensity per Capita (2015)" DESC;
```

Resultado de la Consulta | Salida de Script | Salida de DBMS | Explicación del plan | Rastreo automático | Historial SQL

Descargar | Tiempo de ejecución: 0,021 segundos

	COUNTRY	ENERGY INTENSITY PER
1	Qatar	776.4352134790557
2	Singapore	593.5897261540538
3	United Arab Emira...	507.63797572439...
4	Bahrain	506.99095266069...
5	Kuwait	436.99295187607...
6	Canada	411.89050530323...
7	Brunei	402.65523992538...
8	Norway	370.66281016568...
9	Saudi Arabia	343.3657598460123
10	Turkmenistan	315.95064792966...
11	Luxembourg	309.9130586502779
12	United States	302.8191490494935
13	Oman	282.67373709020...
14	Australia	242.1599504052642
15	South Korea	234.40469422379...
16	Netherlands	224.96124349485...
17	Malta	221.82167382091...
18	Belgium	221.0788236271336
19	Sweden	219.6670378433108
20	Finland	217.78446533374...
21	Russia	213.41925070577...

72	Latvia	80.14937128198946
73	Croatia	80.01189879674038

7.11.1 Explanation

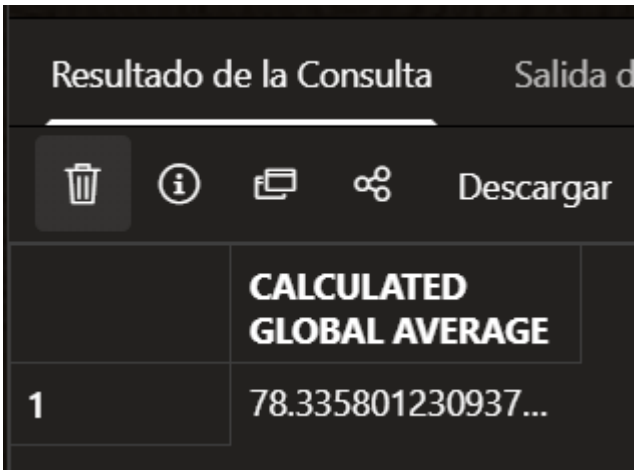
This query identifies nations whose energy intensity per capita strictly exceeded the globally calculated mathematical average during the year 2015. To accomplish this dynamically, a scalar subquery is deployed within the WHERE clause to compute the true AVG(energy_intensity_per_capita) baseline for that specific temporal period.

7.11.2 Validation

We must first calculate the exact numerical threshold generated by the subquery behind the scenes.

Croatia with 80.01 is the last valute that fits within the query.

```sql
SELECT
    AVG(energy_intensity_per_capita) AS "Calculated Global Average 2015"
FROM
    Yearly_Stats
WHERE
    year_number = 2015
    AND country_name != 'World';
```



7.12 (Which countries have reduced their CO_2 emissions from electricity and heat between 2015 and 2019?)

```
SELECT

    e2015.country_name AS "Country",

    e2015.co2_from_electricity_and_heat AS "Emissions (2015)",

    e2019.co2_from_electricity_and_heat AS "Emissions (2019)"

FROM

    Yearly_Emits e2015

JOIN

    Yearly_Emits e2019

    ON e2015.country_name = e2019.country_name

WHERE

    e2015.year_number = 2015

    AND e2019.year_number = 2019

    AND e2015.country_name != 'World'

    AND e2019.co2_from_electricity_and_heat < e2015.co2_from_electricity_and_heat

ORDER BY

    "Country" ASC;
```

Resultado de la Consulta | Salida de Script | Salida de DBMS | Explicación del

Descargar | Tiempo de ejecución: 0,019 segundos

	COUNTRY	EMISSIONS (2015)	EMISSIONS (2019)
1	Angola	6420000	5740000
2	Argentina	69720000	56690000
3	Austria	19390000	18810000
4	Azerbaijan	14340000	14190000
5	Barbados	640000	500000
6	Bolivia	4740000	4500000
7	Brazil	122110000	92360000
8	Bulgaria	29300000	22510000
9	Burkina Faso	580000	410000
10	Canada	216230000	207370000
11	Colombia	21850000	19180000
12	Costa Rica	100000	70000
13	Cote d'Ivoire	3960000	3830000
14	Croatia	4670000	3850000
15	Cuba	16750000	14180000
16	Czechia	58560000	53700000
17	Democratic Repub...	10000	0
18	Denmark	12780000	8570000
19	Djibouti	140000	50000
20	Dominica	100000	90000
21	Dominican Republic	10800000	10750000
22	Ecuador	10470000	6910000

7.12.1 Explanation

This query identifies sovereign Countries that successfully reduced their CO_2 emissions within the electricity and heat sector between the years 2015 and 2019. To achieve this temporal comparison, a 'Self-Join' architecture is implemented, aliasing the Yearly_Emits table twice (e2015 and e2019) and linking the records via the country_name attribute. These specific years and metrics were selected following a data profiling phase that confirmed high data integrity for these periods. The 'World' aggregate row is intentionally excluded to prevent skewed macro-level comparisons. Finally, the conditional logic e2019.co2_from_electricity_and_heat < e2015.co2_from_electricity_and_heat is applied in the WHERE clause to exclusively extract nations demonstrating a verified ecological reduction,

7.12.2 Validation

We test whether the query correctly excluded countries that actually increased their emissions by intentionally reversing the comparative operator.

```
SELECT

    e2015.country_name,

    e2015.co2_from_electricity_and_heat AS "Emissions 2015",

    e2019.co2_from_electricity_and_heat AS "Emissions 2019"

FROM

    Yearly_Emits e2015

JOIN

    Yearly_Emits e2019

    ON e2015.country_name = e2019.country_name

WHERE

    e2015.year_number = 2015

    AND e2019.year_number = 2019

    AND e2015.country_name != 'World'

    AND e2019.co2_from_electricity_and_heat > e2015.co2_from_electricity_and_heat

;
```

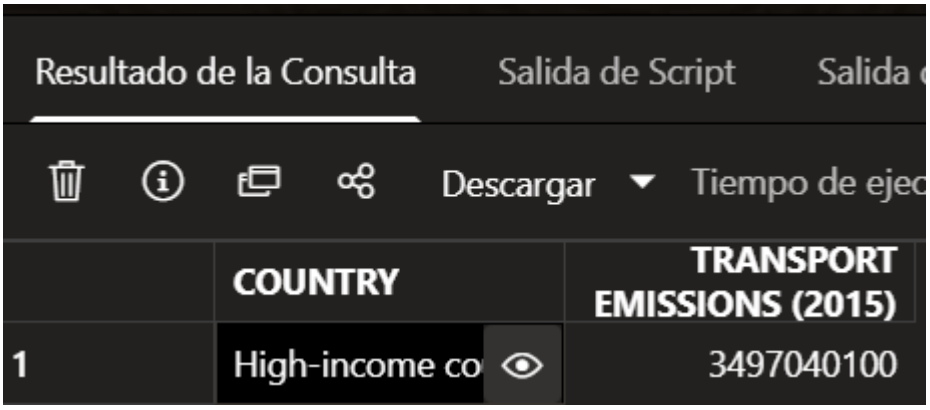
Resultado de la Consulta | Salida de Script | Salida de DBMS | Explicación del plan | Rastr

Descargar | Tiempo de ejecución: 0,007 segundos

	COUNTRY_NAME	EMISSIONS 2015	EMISSIONS 2019
1	South Korea	351340000	360490000
2	Saudi Arabia	261040020	262739980
3	Uganda	60000	80000
4	Zambia	330000	2380000
5	Armenia	1290000	1360000
6	Asia	8511269400	9837929000
7	Belize	20000	70000
8	Afghanistan	170000	240000
9	Algeria	49760000	50030000
10	Congo	1260000	2130000
11	Gabon	910000	1520000
12	Gambia	240000	259999.98
13	Grenada	129999.99	170000
14	Guyana	790000	930000
15	Georgia	1260000	1350000
16	Niger	519999.97	590000
17	Morocco	22230000	28850000
18	Philippines	49470000	72710000

7.13 (Which country has the highest CO_2 emissions from transport in 2015?)

```
SELECT
    country_name AS "Country",
    co2_from_transport AS "Transport Emissions (2015)"
FROM
    Yearly_Emits
WHERE
    year_number = 2015
    AND country_name != 'World'
    AND co2_from_transport IS NOT NULL
ORDER BY
    co2_from_transport DESC
FETCH FIRST 1 ROWS ONLY;
```



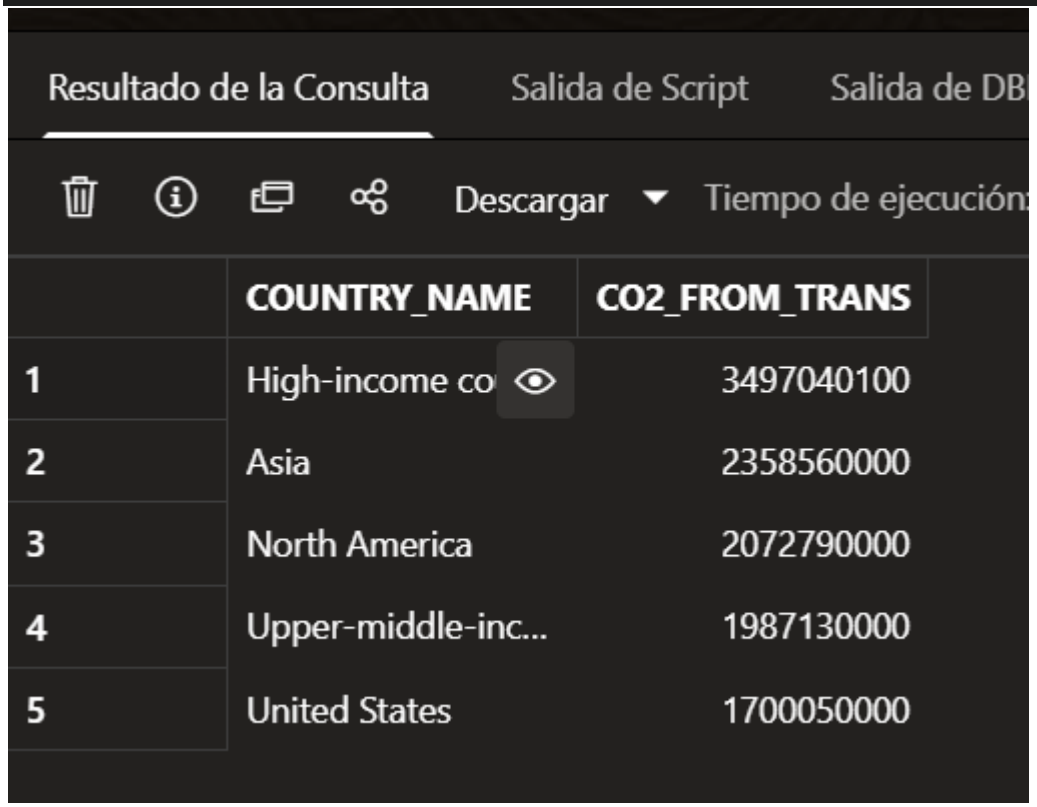
7.13.1 Explanation

This query identifies the entity responsible for the highest volume of CO_2 emissions within the transportation sector during the year 2015. While the 'World' aggregate row was intentionally excluded to avoid obvious global sum totals, the dataset's native schema also categorizes broad socio-economic groupings such as 'High-income countries'. Consequently, the query mathematically isolated this specific high-income bloc as the primary emitter, accurately reflecting the raw dataset's structural hierarchy without artificially altering the original data source. An IS NOT NULL condition was enforced to prevent empty records from defaulting to the top during the ORDER BY ... DESC sort, and the FETCH FIRST 1 ROWS ONLY clause was utilized to accurately truncate the dataset at the absolute maximum recorded value.

7.13.2 Validation

We retrieve the top 5 results to observe how economic blocs compare against the highest individual sovereign states.

```
SELECT
    country_name,
    co2_from_transport
FROM
    Yearly_Emits
WHERE
    year_number = 2015
    AND country_name != 'World'
    AND co2_from_transport IS NOT NULL
ORDER BY
    co2_from_transport DESC
FETCH FIRST 5 ROWS ONLY;
```



	COUNTRY_NAME	CO2_FROM_TRANS
1	High-income co	3497040100
2	Asia	2358560000
3	North America	2072790000
4	Upper-middle-inc...	1987130000
5	United States	1700050000

7.14 (Which countries have CO_2 emissions from industry above the average of all countries in 2010?)

```
SELECT
    country_name AS "Country",
    co2_from_industry AS "Industry Emissions (2010)"
FROM
    Yearly_Emits
WHERE
    year_number = 2010
    AND country_name != 'World'
    AND co2_from_industry > (
        SELECT AVG(co2_from_industry)
        FROM Yearly_Emits
        WHERE year_number = 2010 AND country_name != 'World'
    )
ORDER BY
    co2_from_industry DESC;
```



Resultado de la Consulta | Salida de Script | Salida de DBMS

Descargar | Tiempo de ejecución: 0,0

	COUNTRY	INDUSTRY EMISSIONS (2010)
1	Asia	966779970
2	Upper-middle-inc...	797710000
3	China	639600000
4	Lower-middle-inc...	230670000
5	High-income coun...	208830000
6	Europe	113660000
7	India	86030000
8	European Union (...	76960000
9	North America	61470000
10	Africa	59670000
11	South America	40350000
12	United States	31450000
13	Turkey	29980000
14	Japan	24320000
15	South Korea	24300000
16	Vietnam	23770000
17	Russia	22720000
18	Iran	21560000
19	Brazil	21290000
20	Saudi Arabia	20990000
21	Egypt	20510000
22	Indonesia	19460000

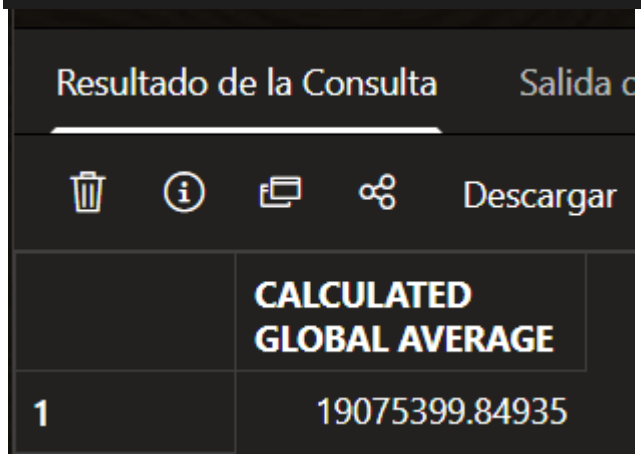
7.14.1 Explanation

This query identifies entities whose industrial CO_2 emissions strictly exceeded the global mathematical average during the year 2010. To calculate this threshold dynamically, a scalar subquery is embedded within the WHERE clause, evaluating the AVG(co2_from_industry) for that specific temporal period rather than relying on a static, hardcoded figure. A critical design choice is the explicit exclusion of the pre-aggregated 'World' row (country_name != 'World') in both the primary query and the subquery, ensuring the arithmetic mean is not artificially skewed by macro-level totals. Furthermore, empty or null records are inherently filtered out by the mathematical > operator.

7.14.2 Validation

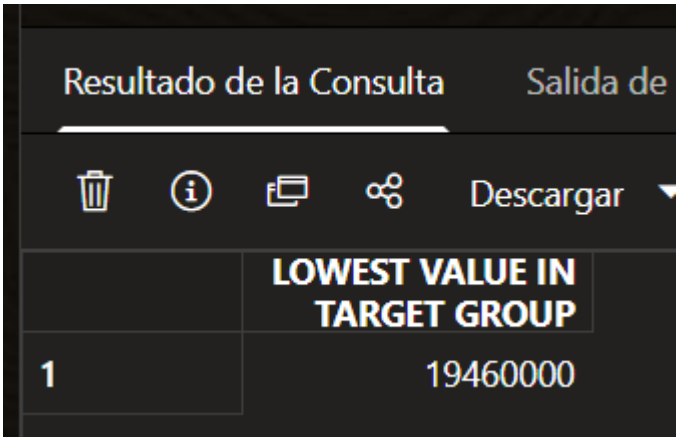
We must first calculate the exact numerical threshold generated by the subquery to establish our mathematical baseline for the year 2010.

```sql
-- Control Query 1: Determine the exact global average for Industry in 2010
SELECT
    AVG(co2_from_industry) AS "Calculated Global Average (2010)"
FROM
    Yearly_Emits
WHERE
    year_number = 2010
    AND country_name != 'World';
```



Next, we isolate the minimum industrial emission value present in our primary query's output to prove it successfully bypassed the calculated threshold.

```sql
SELECT
    MIN(co2_from_industry) AS "Lowest Value in Target Group"
FROM
    Yearly_Emits
WHERE
    year_number = 2010
    AND country_name != 'World'
    AND co2_from_industry > (SELECT AVG(co2_from_industry) FROM Yearly_Emits WHERE
year_number = 2010 AND country_name != 'World');
```



By manually verifying that the numerical result of Test 2 is strictly greater than the result of Test 1, we empirically substantiate that the subquery filter executed flawlessly.

7.15 (What is the total CO_2 emission from all sectors per country in 2015?)

```
SELECT

    country_name AS "Country",

    (co2_from_electricity_and_heat + co2_from_transport + co2_from_industry) AS "Total
Sector Emissions (2015)"

FROM

    Yearly_Emits

WHERE

    year_number = 2015

    AND country_name != 'World'

    AND country_name != 'High-income countries'

    AND co2_from_electricity_and_heat IS NOT NULL

    AND co2_from_transport IS NOT NULL

    AND co2_from_industry IS NOT NULL

ORDER BY

    "Total Sector Emissions (2015)" DESC;
```



	COUNTRY	TOTAL SECTOR EMISSIONS (2015)
1	Asia	11999669400
2	Upper-middle-inc...	9913480050
3	China	6144219700
4	North America	4852130100
5	United States	3973980000
6	Europe	3733270000
7	Lower-middle-inc...	3400520030
8	European Union (...	2026560030
9	India	1478370000
10	Russia	1142260000
11	Africa	975120000
12	Japan	844919970
13	South America	804859970
14	Germany	516480000
15	South Korea	475520000
16	Saudi Arabia	437650020
17	Canada	402240000

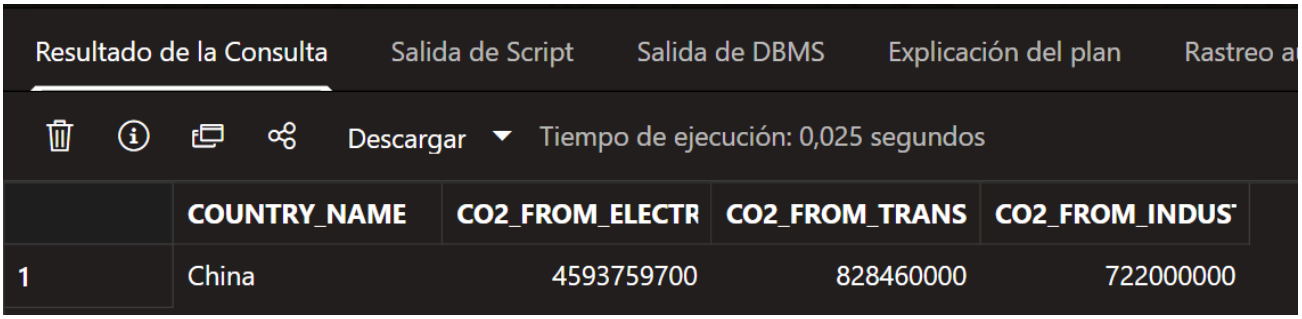
7.15.1 Explanation

This query calculates the comprehensive CO_2 emissions per entity for the year 2015 by performing horizontal row-level arithmetic (+) across the primary sector attributes. To maintain mathematical integrity without relying on advanced null-handling functions, strict IS NOT NULL conditions are enforced in the WHERE clause for every sector. This guarantees that only complete records are evaluated, preventing empty values from corrupting the horizontal addition. Furthermore, basic inequality operators (!=) are chained using AND to manually exclude macro-regions like 'World' and 'High-income countries' from skewing the results.

7.15.2 Validation

We extract the raw, uncalculated sector columns for a specific major country (e.g., 'United States' or 'China') to manually verify the engine's mathematical summation.

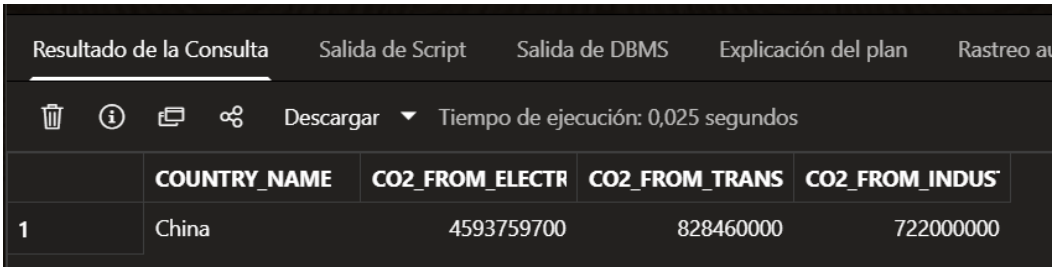
```
SELECT
    country_name,
    co2_from_electricity_and_heat,
    co2_from_transport,
    co2_from_industry
FROM
    Yearly_Emits
WHERE
    year_number = 2015
    AND country_name = 'China';
```



	COUNTRY_NAME	CO2_FROM_ELECTR	CO2_FROM_TRANS	CO2_FROM_INDUS
1	China	4593759700	828460000	722000000

7.16 (Which countries have higher CO_2 emissions from electricity and heat than from transport in 2015?)

```sql
SELECT
    country_name AS "Country",
    co2_from_electricity_and_heat AS "Electricity & Heat Emissions",
    co2_from_transport AS "Transport Emissions"
FROM
    Yearly_Emits
WHERE
    year_number = 2015
    AND country_name != 'World'
    AND country_name != 'High-income countries'
    AND country_name != 'Upper-middle-income countries'
    AND country_name != 'Lower-middle-income countries'
    AND country_name != 'Low-income countries'
    AND country_name != 'Europe'
    AND country_name != 'Asia'
    AND country_name != 'North America'
    AND country_name != 'South America'
    AND country_name != 'Africa'
    AND country_name != 'Oceania'
    AND country_name != 'European Union (27)'
    AND co2_from_electricity_and_heat IS NOT NULL
    AND co2_from_transport IS NOT NULL
    AND co2_from_electricity_and_heat > co2_from_transport
ORDER BY
    "Country" ASC;
```



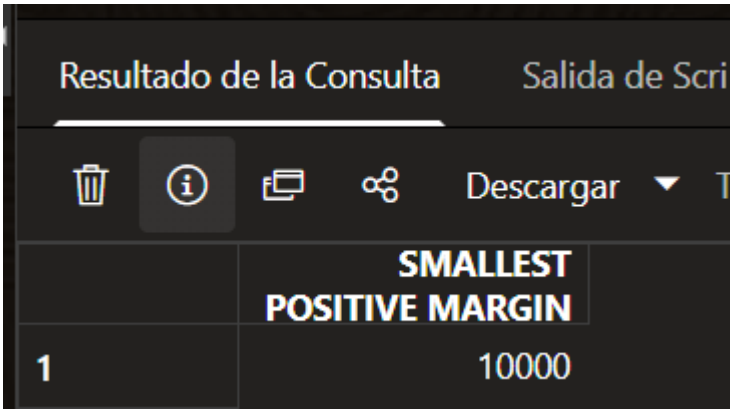
7.16.1 Explanation

This query identifies sovereign states where the carbon footprint from electricity and heat generation strictly exceeded that of the transportation sector during the year 2015. The design utilizes a mathematical comparison (>) between two distinct sector attributes within the same record. To ensure absolute mathematical integrity and prevent the engine from evaluating empty data points, explicit IS NOT NULL conditions are enforced for both sectors. Furthermore, fundamental inequality operators (!=) are chained together using AND to systematically exclude pre-aggregated macro-regions and income brackets from the dataset.

7.16.2 Validation

If Electricity strictly exceeds Transport, the mathematical difference between the two must be strictly greater than zero (> 0). By calculating the absolute minimum difference across our entire filtered dataset, we can prove that not a single row violated our condition.

```sql
SELECT
    MIN(co2_from_electricity_and_heat - co2_from_transport) AS "Smallest Positive Margin"
FROM
    Yearly_Emits
WHERE
    year_number = 2015
    AND country_name != 'World'
    AND country_name != 'Europe'
    AND co2_from_electricity_and_heat IS NOT NULL
    AND co2_from_transport IS NOT NULL
    AND co2_from_electricity_and_heat > co2_from_transport;
```



7.17 (Which city had the worst average PM2.5 level across all recorded dates?)

```sql
SELECT
    CITY,
    AVG(PM2_5) AS "Average PM2.5 Level"
FROM
    GLOBAL_AIR_QUALITY_DATA_10000
WHERE
    PM2_5 IS NOT NULL
GROUP BY
    CITY
ORDER BY
    "Average PM2.5 Level" DESC
FETCH FIRST 1 ROWS ONLY;
```



	CITY	AVERAGE PM2.5 LEVEL
1	Dubai	80.013692307692...

7.17.1 Explanation

This query identifies the specific municipality that recorded the highest average concentration of PM2.5 particulate matter across all available temporal records. To achieve this, the dataset from the GLOBAL_AIR_QUALITY_DATA_10000 table is clustered using the GROUP BY clause on the CITY attribute, while the AVG() function mathematically computes the mean pollution level for each respective location. An IS NOT NULL constraint is proactively applied in the WHERE clause to ensure the arithmetic mean is calculated exclusively from valid data points, protecting the aggregation from null-induced logical corruption. Finally, the aggregated results are sorted in descending sequence and truncated using the FETCH FIRST 1 ROWS ONLY clause to definitively isolate the single most polluted urban center.

7.17.2 Validation

We manually extract the raw, unaggregated historical records for the #1 city returned by our main query (e.g., 'Dubai' or whichever city appeared first) to prove the arithmetic mean was computed perfectly.

```
SELECT
    CITY,
    PM2_5
FROM
    GLOBAL_AIR_QUALITY_DATA_10000
WHERE
    CITY = 'Dubai'
    AND PM2_5 IS NOT NULL;
```

Resultado de la Consulta | Salida de Script | Salida de DBMS

Descargar | Tiempo de ejecución: 0,008

	CITY	PM2_5
1	Dubai	58.2
2	Dubai	142.35
3	Dubai	67.6
4	Dubai	114.11
5	Dubai	80.56
6	Dubai	115.42
7	Dubai	99.67
8	Dubai	36.24
9	Dubai	103.53
10	Dubai	7.9
11	Dubai	48.53
12	Dubai	22.27
13	Dubai	113.57
14	Dubai	64.28
15	Dubai	64.81
16	Dubai	115.11
17	Dubai	59.32
18	Dubai	80.09
19	Dubai	148.59
20	Dubai	90.17
21	Dubai	123.42
22	Dubai	137.86

7.18 (Which countries have an average NO_2 level above 50 across all their cities?)

```
SELECT c.country_name AS "Country Name", AVG(r.no2) AS "Average NO2 Level"

FROM City c

JOIN Registers r ON c.city_name = r.city_name

GROUP BY c.country_name

HAVING AVG(r.no2) > 50

ORDER BY "Average NO2 Level" DESC;
```

7.18.1 Explanation

This query finds the countries whose cities have an average NO_2 level greater than 50. It joins City with Register using city_name, because air quality measurements are stored per city and date in Register, while the country of each city is stored in City. The query groups the records by country and calculates the average NO2 value for all measurements from all cities in each country. The HAVING clause filters only those countries where the average NO_2 level is above 50.

7.17.2 Validation

This validation query checks the same grouped result as the main query, but it also displays the number of records available for each country. COUNT(*) shows the total number of measurements stored for the country, while COUNT(r.no2) shows how many of those records actually contain a valid NO_2 value. This is important because the Registers table contains repeated measurements for the same cities, and some NO_2 values may be null. The average is therefore calculated only from the non-null NO_2 records used by Oracle in AVG(r.no2).

```
SELECT r.country_name AS "Country Name", COUNT(*) AS "Total Records", COUNT(r.no2) AS
"NO2 Records Used", AVG(r.no2) AS "Average NO2 Level"

FROM Registers r

GROUP BY r.country_name

HAVING AVG(r.no2) > 50

ORDER BY "Average NO2 Level" DESC;
```

Query Result | Script Output | DBMS Output | Explain Plan | Autotrace | SQL History

Download | Execution time: 0.043 seconds

	COUNTRY NAME	TOTAL RECORDS	NO2 RECORDS USED	AVERAGE NO2 LEVEL
1	India	37326	267	67.374756554307...
2	UAE	252	252	65.135674603174...
3	South Africa	9536	271	64.861402214022...
4	Australia	46573	248	64.691774193548...
5	Germany	23122	261	64.057739463601...
6	Spain	273	273	63.768901098901...
7	USA	513	513	63.710545808966...
8	South Korea	9521	256	63.6623046875
9	France	18795	265	63.476490566037...
10	Japan	28052	257	63.301478599221...
11	UK	255	255	62.912549019607...
12	Turkey	259	259	62.446602316602...
13	Brazil	256	256	62.3973828125

7.19 (Which cities have an average humidity above 80% and an average PM10 above 100?)

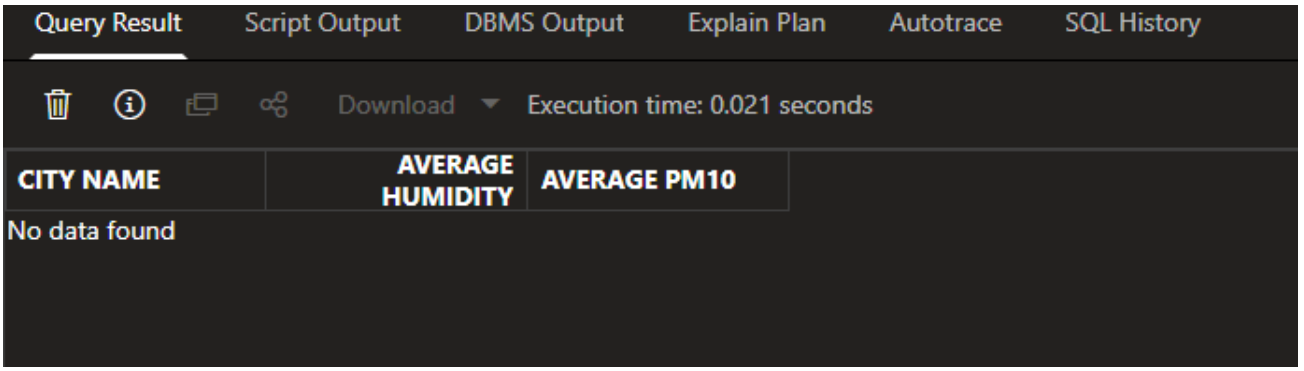
```
SELECT r.city_name AS "City Name", AVG(r.humidity) AS "Average Humidity", AVG(r.pm10) AS
"Average PM10"

FROM Registers r

GROUP BY r.city_name

HAVING AVG(r.humidity) > 80 AND AVG(r.pm10) > 100

ORDER BY "Average Humidity" DESC, "Average PM10" DESC;
```



7.19.1 Explanation

This query groups the records in Registers by city and calculates the average humidity and PM10 level for each city. The HAVING clause filters the result to show only cities where average humidity is above 80% and average PM10 is above 100.

7.19.2 Validation

This validation query shows the individual humidity and PM10 measurements for the cities returned by the main query, allowing us to verify the values used to calculate the averages.

```
SELECT r.city_name AS "City Name", r.humidity AS "Humidity", r.pm10 AS "PM10"

FROM Registers r

WHERE r.city_name IN (

    SELECT r2.city_name

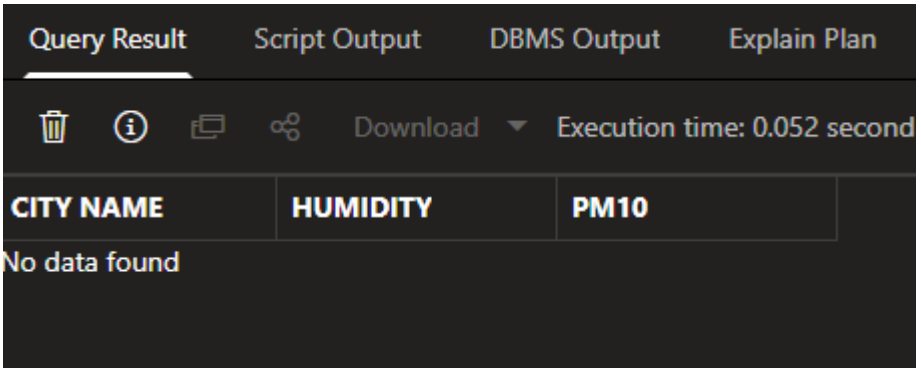
    FROM Registers r2

    GROUP BY r2.city_name

    HAVING AVG(r2.humidity) > 80 AND AVG(r2.pm10) > 100

)

ORDER BY r.city_name ASC, r.humidity DESC, r.pm10 DESC;
```


Query Result
Script Output
DBMS Output
Explain Plan
Download
Execution time: 0.052 second
CITY NAME
HUMIDITY
PM10
No data found

7.20 (Which city had the highest average temperature recorded in the air quality data?)

```
SELECT r.city_name AS "City Name", AVG(r.temperature) AS "Average Temperature"

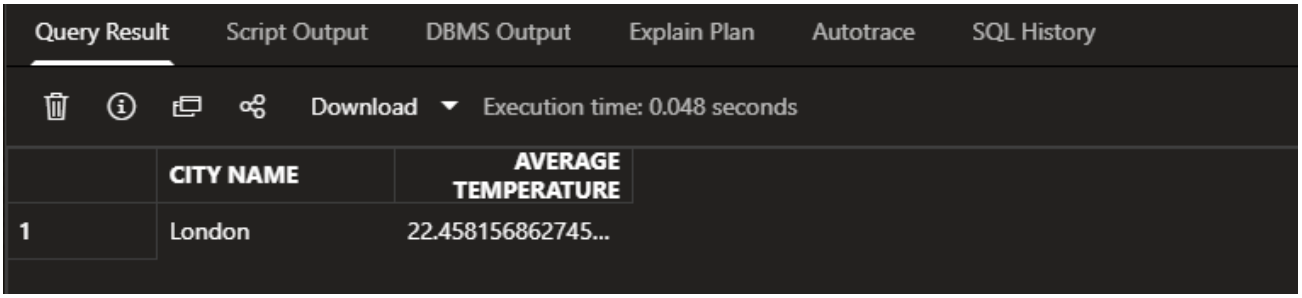
FROM Registers r

WHERE r.temperature IS NOT NULL

GROUP BY r.city_name

ORDER BY "Average Temperature" DESC

FETCH FIRST 1 ROW ONLY;
```



7.20.1 Explanation

This query identifies the city with the highest average recorded temperature in the Registers table. The query groups all temperature measurements by city and calculates the average temperature for each one using AVG(r.temperature). The WHERE clause excludes null temperature values to ensure that only valid measurements are considered. The results are ordered from the highest average temperature to the lowest, and only the first row is returned.

7.20.2 Validation

This validation query displays the average temperature for every city together with the number of valid temperature records used in the calculation. It also shows the maximum and minimum recorded temperatures for each city, allowing us to verify that the city returned in the main query truly has the highest average temperature among all cities in the dataset.

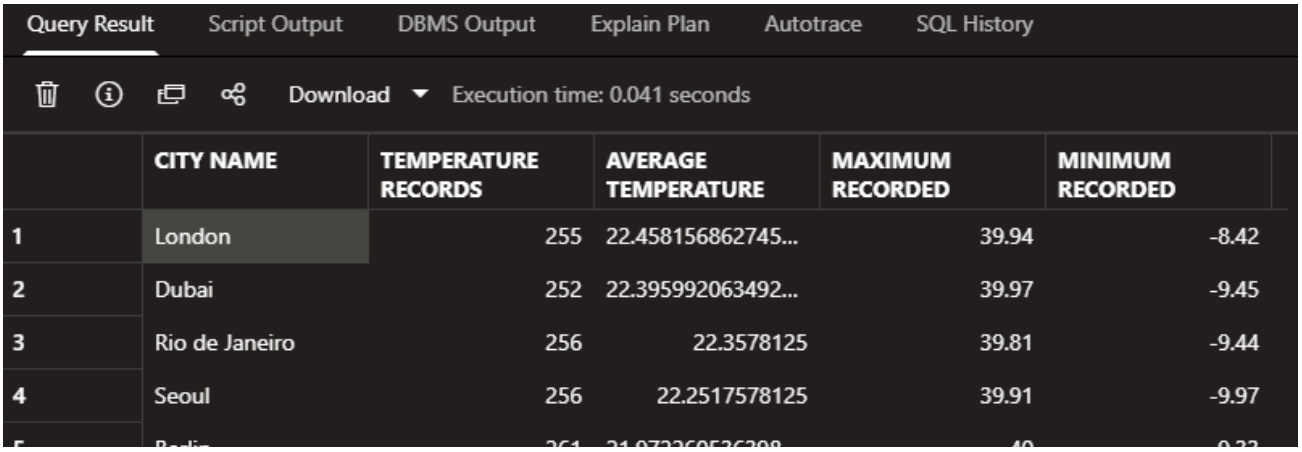
```
SELECT r.city_name AS "City Name", COUNT(r.temperature) AS "Temperature Records",
AVG(r.temperature) AS "Average Temperature", MAX(r.temperature) AS "Maximum Recorded
Temperature", MIN(r.temperature) AS "Minimum Recorded Temperature"

FROM Registers r

WHERE r.temperature IS NOT NULL

GROUP BY r.city_name

ORDER BY "Average Temperature" DESC;
```



	CITY NAME	TEMPERATURE RECORDS	AVERAGE TEMPERATURE	MAXIMUM RECORDED	MINIMUM RECORDED
1	London	255	22.458156862745...	39.94	-8.42
2	Dubai	252	22.395992063492...	39.97	-9.45
3	Rio de Janeiro	256	22.3578125	39.81	-9.44
4	Seoul	256	22.2517578125	39.91	-9.97

7.21 (Which city has the highest average temperature in July across all recorded years?)

```
SELECT r.city_name AS "City Name", AVG(r.temperature) AS "Average July Temperature"

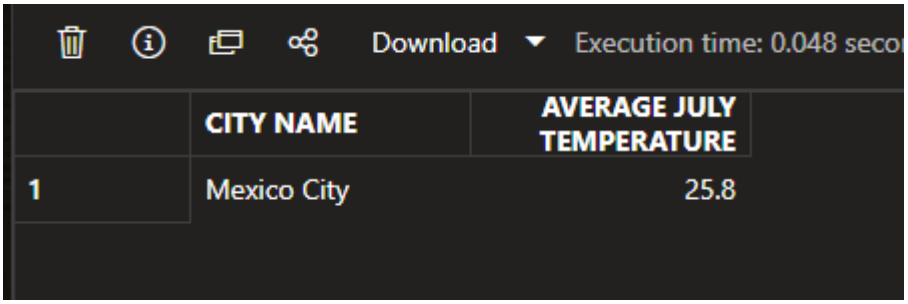
FROM Registers r

WHERE r.temperature IS NOT NULL AND r.month_number = 7

GROUP BY r.city_name

ORDER BY "Average July Temperature" DESC

FETCH FIRST 1 ROW ONLY;
```



	CITY NAME	AVERAGE JULY TEMPERATURE
1	Mexico City	25.8

7.21.1 Explanation

This query identifies the city with the highest average temperature during the month of July across all recorded years. The WHERE clause filters the dataset to include only records where month_number = 7, corresponding to July, and excludes null temperature values. The query then groups the records by city and calculates the average temperature for each city using AVG(r.temperature). Finally, the results are sorted in descending order, and only the city with the highest average July temperature is returned.

7.21.2 Validation

This validation query displays the average July temperature for every city together with the number of valid July temperature records used in the calculation. It also includes the maximum and minimum July temperatures recorded for each city. This allows verification that the city returned by the main query truly has the highest average July temperature in the dataset.

```
SELECT r.city_name AS "City Name", COUNT(r.temperature) AS "July Temperature Records",
AVG(r.temperature) AS "Average July Temperature", MAX(r.temperature) AS "Maximum July
Temperature", MIN(r.temperature) AS "Minimum July Temperature"

FROM Registers r

WHERE r.temperature IS NOT NULL AND r.month_number = 7

GROUP BY r.city_name

ORDER BY "Average July Temperature" DESC;
```

Query Result | Script Output | DBMS Output | Explain Plan | Autotrace | SQL History

Download | Execution time: 0.044 seconds

	CITY NAME	JULY TEMPERATURE	AVERAGE JULY TEMPERATURE	MAXIMUM JULY TEMPERATURE	MINIMUM JULY TEMPERATURE
1	Mexico City	21	25.8	38.57	7.17
2	Berlin	24	25.514166666666...	37.6	1.93
3	Dubai	19	25.373684210526...	38.9	-4.12
4	Rio de Janeiro	22	24.258636363636...	37.92	-5.26

7.22 (Which country has the highest average temperature in winter (December, January, February?)

```
SELECT r.country_name AS "Country Name", AVG(r.temperature) AS "Average Winter
Temperature"

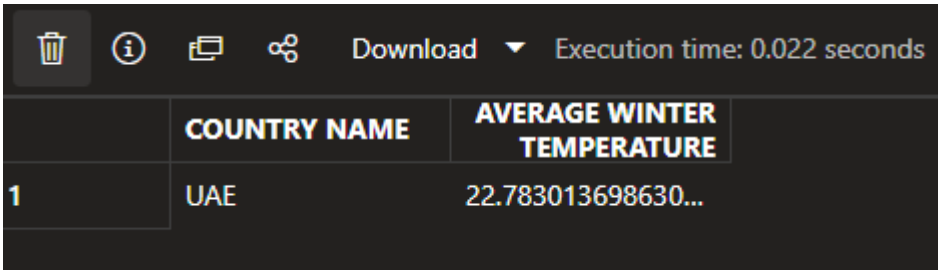
FROM Registers r

WHERE r.temperature IS NOT NULL AND r.month_number IN (12, 1, 2)

GROUP BY r.country_name

ORDER BY "Average Winter Temperature" DESC

FETCH FIRST 1 ROW ONLY;
```



7.22.1 Explanation

This query identifies the country with the highest average temperature during the winter months: December, January, and February. The WHERE clause filters the dataset to include only records corresponding to those months and excludes null temperature values. The query groups the records by country and calculates the average winter temperature using AVG(r.temperature). The results are ordered from the highest average temperature to the lowest, and only the top country is returned.

7.22.2 Validation

This validation query displays the average winter temperature for every country together with the number of valid winter temperature records used in the calculation. It also shows the maximum and minimum temperatures recorded during winter months. This allows verification that the country returned by the main query truly has the highest average winter temperature in the dataset.

```
SELECT r.country_name AS "Country Name", COUNT(r.temperature) AS "Winter Temperature
Records", AVG(r.temperature) AS "Average Winter Temperature", MAX(r.temperature) AS
"Maximum Winter Temperature", MIN(r.temperature) AS "Minimum Winter Temperature"

FROM Registers r

WHERE r.temperature IS NOT NULL AND r.month_number IN (12, 1, 2)

GROUP BY r.country_name

ORDER BY "Average Winter Temperature" DESC;
```

Query Result | Script Output | DBMS Output | Explain Plan | Autotrace | SQL History

Download | Execution time: 0.022 seconds

	COUNTRY NAME	WINTER TEMPERATURE	AVERAGE WINTER TEMPERATURE	MAXIMUM WINTER	MINIMUM WINTER
1	UAE	73	22.783013698630...	39.87	-7.44
2	UK	61	22.774918032786...	39.94	-5.76
3	Mexico	63	22.655238095238...	39.16	-9.27
4	Germany	58	22.410517241379...	39.96	-7.39
5	Thailand	67	22.257910447761...	39.48	-9.34

7.23 (Which cities have a cost of living index above the average cost of living index of all cities?)

```
SELECT c.city_name AS "City Name", c.country_name AS "Country Name",
c.cost_of_living_index AS "Cost Of Living Index"

FROM City c

WHERE c.cost_of_living_index > (

    SELECT AVG(c2.cost_of_living_index)

    FROM City c2

    WHERE c2.cost_of_living_index IS NOT NULL

)

ORDER BY c.cost_of_living_index DESC;
```

	CITY NAME	COUNTRY NAME	COST OF LIVING INDEX
30	Bonn	Germany	69.14
31	Rome	Italy	69.03
32	Berlin	Germany	67.75
33	Taipei	Taiwan	65.75
34	Nicosia	Cyprus	60.97
35	Athens	Greece	60.44
36	Madrid	Spain	60.12

7.23.1 Explanation

This query identifies cities whose cost of living index is higher than the average cost of living index across all cities. The subquery calculates the global average using all non-null cost_of_living_index values from the City table. The main query then returns only the cities whose index is above that average.

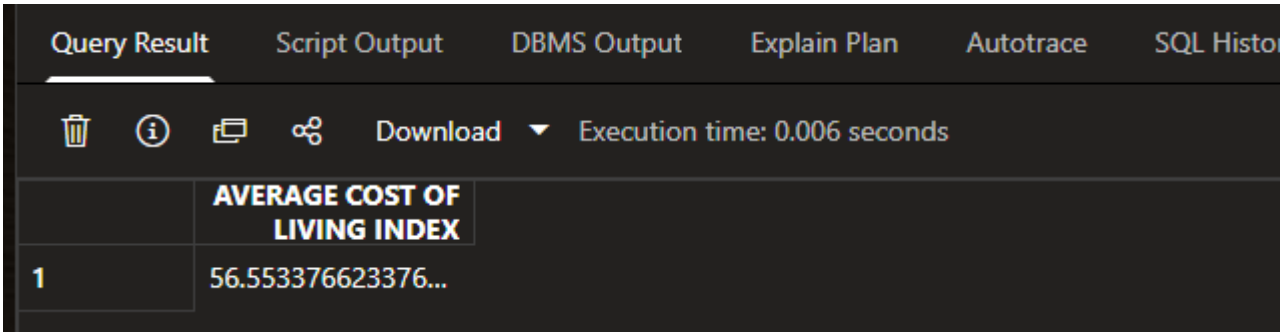
7.23.2 Validation

This validation query returns the average cost of living index used as the threshold in the main query. Every city returned by the main query should have a cost_of_living_index greater than this value.

```
SELECT AVG(c.cost_of_living_index) AS "Average Cost Of Living Index"

FROM City c

WHERE c.cost_of_living_index IS NOT NULL;
```


Query Result
Script Output
DBMS Output
Explain Plan
Autotrace
SQL Histo
Download
Execution time: 0.006 seconds
AVERAGE COST OF LIVING INDEX
1
56.553376623376...

7.24 (Which cities have a cost of living index above average and belong to countries with high NO_2 levels?)

```
SELECT c.country_name AS "Country Name"

FROM City c

WHERE c.country_name IS NOT NULL

MINUS

SELECT gp.country_name AS "Country Name"

FROM Government_Policy gp

WHERE gp.country_name IS NOT NULL

ORDER BY "Country Name";
```

	COUNTRY NAME
1	Ivory Coast
2	Macedonia
3	Myanmar (Burma)
4	Taiwan
5	The Netherlands
6	UAE
7	UK
8	USA

7.24.1 Explanation

This query uses MINUS, the Oracle equivalent of EXCEPT, to identify countries that appear in the City table but do not appear in Government_Policy. The first query selects countries with city-level socioeconomic data, while the second query removes countries that have at least one recorded government policy. This helps detect countries with urban data but missing policy information.

7.24.2 Validation

This validation checks the same logic using a LEFT JOIN. If a country appears in the main query, it should have zero matching rows in Government_Policy. Therefore, every country returned by the validation must show Total Policies = 0, confirming that the MINUS operation correctly excluded countries with policies.

```
SELECT c.country_name AS "Country Name", COUNT(gp.policy_id) AS "Total Policies"
FROM City c
LEFT JOIN Government_Policy gp ON c.country_name = gp.country_name
GROUP BY c.country_name
HAVING COUNT(gp.policy_id) = 0
ORDER BY c.country_name;
```

	COUNTRY NAME
1	Ivory Coast
2	Macedonia
3	Myanmar (Burma)
4	Taiwan
5	The Netherlands
6	UAE
7	UK
8	USA

7.25 (What are the top 5 cities with the highest local purchasing power index?)

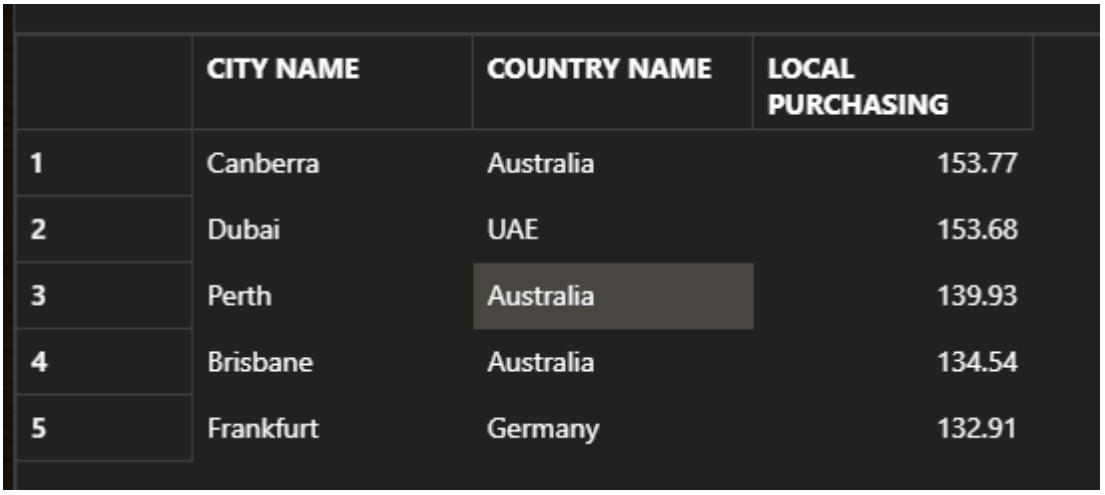
```
SELECT c.city_name AS "City Name", c.country_name AS "Country Name",
c.local_purchasing_power_index AS "Local Purchasing Power Index"

FROM City c

WHERE c.local_purchasing_power_index IS NOT NULL

ORDER BY c.local_purchasing_power_index DESC

FETCH FIRST 5 ROWS ONLY;
```



	CITY NAME	COUNTRY NAME	LOCAL PURCHASING
1	Canberra	Australia	153.77
2	Dubai	UAE	153.68
3	Perth	Australia	139.93
4	Brisbane	Australia	134.54
5	Frankfurt	Germany	132.91

7.25.1 Explanation

This query returns the five cities with the highest local purchasing power index. It filters out null values, orders the cities from highest to lowest purchasing power, and limits the result to the first five rows.

7.25.2 Validation

This validation query lists all cities ordered by local purchasing power index from highest to lowest. The first five rows of this result should match the cities returned by the main query.

```
SELECT c.city_name AS "City Name", c.country_name AS "Country Name",
c.local_purchasing_power_index AS "Local Purchasing Power Index"

FROM City c

WHERE c.local_purchasing_power_index IS NOT NULL

ORDER BY c.local_purchasing_power_index DESC;
```

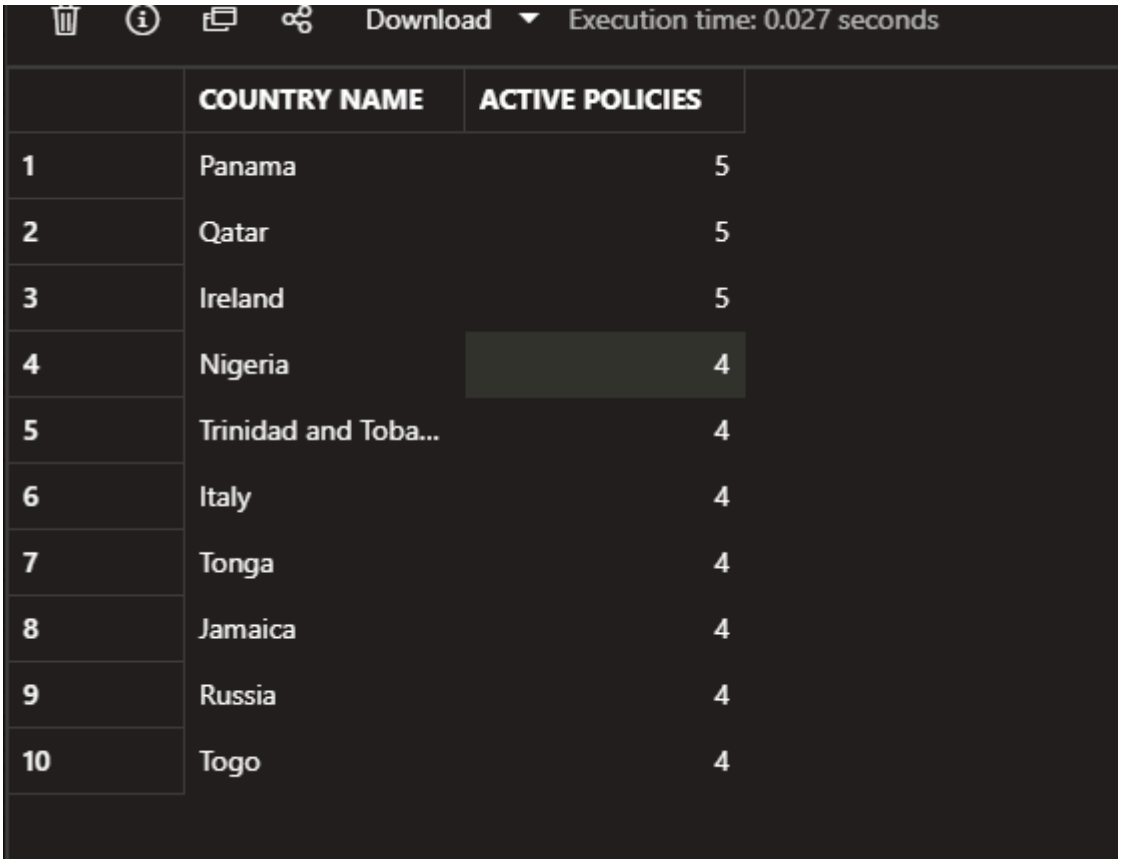
Query Result | Script Output | DBMS Output | Explain Plan | Autotrace | SQL History

Download | Execution time: 0.006 seconds

	CITY NAME	COUNTRY NAME	LOCAL PURCHASING
1	Canberra	Australia	153.77
2	Dubai	UAE	153.68
3	Perth	Australia	139.93
4	Brisbane	Australia	134.54
5	Frankfurt	Germany	132.91
6	Melbourne	Australia	131.85
7	Sydney	Australia	128.25
8	London	UK	118.79
9	Munich	Germany	114.57
10	Los Angeles	USA	114.51

7.26 (Which countries have more than 3 active government policies?)

```
SELECT gp.country_name AS "Country Name", COUNT(*) AS "Active Policies"
FROM Government_Policy gp
WHERE gp.policy_status = 'Active'
GROUP BY gp.country_name
HAVING COUNT(*) > 3
ORDER BY "Active Policies" DESC;
```



	COUNTRY NAME	ACTIVE POLICIES
1	Panama	5
2	Qatar	5
3	Ireland	5
4	Nigeria	4
5	Trinidad and Toba...	4
6	Italy	4
7	Tonga	4
8	Jamaica	4
9	Russia	4
10	Togo	4

7.26.1 Explanation

This query identifies countries that have more than three active government policies. It filters the Government_Policy table to include only policies with status Active, groups the records by country, and counts how many active policies each country has. The HAVING clause keeps only countries with more than three active policies.

7.26.2 Validation

This validation query lists all active policies by country. It allows checking manually that the countries returned by the main query are the ones with more than three active policies.

```
SELECT gp.country_name AS "Country Name", gp.policy_id AS "Policy ID", gp.policy_name AS "Policy Name", gp.policy_status AS "Policy Status"
FROM Government_Policy gp
WHERE gp.policy_status = 'Active'
ORDER BY gp.country_name ASC, gp.policy_id ASC;
```

	COUNTRY NAME	POLICY ID	POLICY NAME	POLICY STATUS
1	Afghanistan	53	Emissions Trading ...	Active
2	Albania	8	Clean Air Act	Active
3	Albania	540	Emissions Trading ...	Active
4	Algeria	578	Nuclear Phase-Out	Active
5	Andorra	548	Wind Power Expa...	Active
6	Antigua and Barb...	69	Wind Power Expa...	Active
7	Argentina	173	Green New Deal	Active
8	Armenia	156	Green New Deal	Active
9	Australia	613	Wind Power Expa...	Active
10	Austria	393	Carbon Tax	Active
11	Bahamas	669	Emissions Trading ...	Active
12	Bangladesh	592	Renewable Energy...	Active
13	Barbados	158	Solar Initiative	Active
14	Belarus	912	Solar Initiative	Active
15	Belize	499	Nuclear Phase-Out	Active
16	Belize	873	Wind Power Expa...	Active

7.27 (Which countries have at least government policy with a budget above the average budget of all policies)

```
SELECT DISTINCT c.country_name

FROM Country c

JOIN Government_Policy gp ON c.country_name = gp.country_name

WHERE gp.budget_million_usd > (

    SELECT AVG(gp2.budget_million_usd)

    FROM Government_Policy gp2

)

ORDER BY c.country_name;
```

	COUNTRY_NAME
1	Afghanistan
2	Albania
3	Algeria
4	Andorra
5	Angola
6	Antigua and Barb...
7	Argentina
8	Armenia
9	Aruba
10	Australia
11	Austria
12	Azerbaijan
13	Bahamas
14	Bahrain

7.27.1 Explanation

This query identifies countries that have at least one government policy whose budget exceeds the global average budget across all policies. A scalar subquery dynamically computes the average budget threshold. DISTINCT is used to avoid duplicate country entries in cases where a country has multiple policies above the threshold.

7.27.2 Validation

The validation query exposes each qualifying policy alongside the country, its budget and the computed

average threshold. Every budget_million_usd value must be strictly greater than avg_budget. The results are ordered descending by budget to easily verify the highest-spending policies at the top.

```
SELECT
    c.country_name,
    gp.name_policy,
    gp.budget_million_usd,
    (SELECT AVG(gp2.budget_million_usd)
     FROM Government_Policy gp2) AS avg_budget
FROM Country c
JOIN Government_Policy gp ON c.country_name = gp.country_name
WHERE gp.budget_million_usd > (
    SELECT AVG(gp2.budget_million_usd)
    FROM Government_Policy gp2
)
ORDER BY gp.budget_million_usd DESC;
```

	COUNTRY_NAME	NAME_POLICY	BUDGET_MILLION_	AVG_BUDGET
1	Honduras	Nuclear Phase-Out	4990	2506.242
2	Yemen	Clean Air Act	4989	2506.242
3	Central African Re...	Nuclear Phase-Out	4987	2506.242
4	Israel	Renewable Energy...	4981	2506.242
5	Gibraltar	Clean Air Act	4979	2506.242
6	Kazakhstan	Emissions Trading ...	4974	2506.242
7	Bahrain	Carbon Tax	4971	2506.242
8	Jamaica	Renewable Energy...	4967	2506.242
9	Russia	Solar Initiative	4959	2506.242
10	Saint Kitts and Ne...	Renewable Energy...	4953	2506.242
11	Canada	Emissions Trading ...	4952	2506.242
12	Equatorial Guinea	Carbon Tax	4940	2506.242
13	Germany	Renewable Energy...	4929	2506.242
14	United Kingdom	Nuclear Phase-Out	4925	2506.242

7.28 (Which countries were both top CO_2 emitters and had active emissions reduction policies in 2015?)

```
SELECT yer.country_name AS "Country Name"

FROM Yearly_Energy_Records yer

WHERE yer.year_number = 2015 AND yer.type_name = 'all_energy_types' AND yer.country_name
!= 'World' AND yer.co2_emission > (

    SELECT AVG(yer2.co2_emission)

    FROM Yearly_Energy_Records yer2

    WHERE yer2.year_number = 2015 AND yer2.type_name = 'all_energy_types' AND
yer2.country_name != 'World')

INTERSECT

SELECT gp.country_name AS "Country Name"

FROM Government_Policy gp

WHERE gp.policy_status = 'Active' AND gp.policy_type = 'Emissions Reduction'

ORDER BY "Country Name";
```

	COUNTRY NAME
1	Argentina
2	China
3	France
4	Germany
5	Indonesia
6	Italy
7	Mexico
8	Singapore
9	Spain
10	United Arab Emira...

7.28.1 Explanation

This query uses INTERSECT to find countries that satisfy two conditions at the same time: having CO_2 emissions above the 2015 average and having an active emissions reduction policy. The first query builds the high-emission country set, and the second one builds the active-policy country set.

7.28.2 Validation

This validation query verifies the same logic using a different SQL structure. Instead of INTERSECT, it uses an IN subquery to check whether each country with an active emissions reduction policy also belongs to the group of above-average CO_2 emitters. The COUNT(gp.policy_id) column confirms that each returned country has at least one active reduction policy. Therefore, the validation proves that the countries returned by the main query satisfy both conditions.

```
SELECT gp.country_name AS "Country Name", COUNT(gp.policy_id) AS "Active Reduction
Policies"

FROM Government_Policy gp

WHERE gp.policy_status = 'Active' AND gp.policy_type = 'Emissions Reduction' AND
gp.country_name IN (

    SELECT yer.country_name

    FROM Yearly_Energy_Records yer

    WHERE yer.year_number = 2015 AND yer.type_name = 'all_energy_types' AND
yer.country_name != 'World' AND yer.co2_emission > (

        SELECT AVG(yer2.co2_emission)

        FROM Yearly_Energy_Records yer2

        WHERE yer2.year_number = 2015 AND yer2.type_name = 'all_energy_types' AND
yer2.country_name != 'World')

)

GROUP BY gp.country_name

ORDER BY gp.country_name;
```

	COUNTRY NAME
1	Argentina
2	China
3	France
4	Germany
5	Indonesia
6	Italy
7	Mexico
8	Singapore
9	Spain
10	United Arab Emira...

7.29 (Which countries appear as extreme cases in 2015, either very high GDP or very high CO_2 emissions?)

```
SELECT ys.country_name AS "Country Name", 'High GDP' AS "Category", ys.gdp AS "Value"
FROM Yearly_Stats ys
WHERE ys.year_number = 2015 AND ys.country_name != 'World' AND ys.gdp > (
    SELECT AVG(ys2.gdp)
    FROM Yearly_Stats ys2
    WHERE ys2.year_number = 2015 AND ys2.country_name != 'World')
UNION ALL
SELECT yer.country_name AS "Country Name", 'High CO2 Emission' AS "Category",
yer.co2_emission AS "Value"
FROM Yearly_Energy_Records yer
WHERE yer.year_number = 2015 AND yer.type_name = 'all_energy_types' AND yer.country_name
!= 'World' AND yer.co2_emission > (
    SELECT AVG(yer2.co2_emission)
    FROM Yearly_Energy_Records yer2
    WHERE yer2.year_number = 2015 AND yer2.type_name = 'all_energy_types' AND
yer2.country_name != 'World')
ORDER BY "Country Name", "Category";
```

	COUNTRY NAME	CATEGORY	VALUE
1	Argentina	High CO2 Emission	197.325871069
2	Argentina	High GDP	867.707
3	Australia	High CO2 Emission	387.062988722
4	Australia	High GDP	1113.001
5	Bangladesh	High GDP	595.349
6	Brazil	High CO2 Emission	509.822374377
7	Brazil	High GDP	3013.558
8	Canada	High CO2 Emission	589.561745673
9	Canada	High GDP	1594.781
10	China	High CO2 Emission	10654.275247466

7.29.1 Explanation

This query uses UNION ALL to combine two groups of countries: those with GDP above the 2015 average and those with CO_2 emissions above the 2015 average. UNION ALL is used because the same country can

appear in both categories, and both reasons are relevant.

7.29.2 Validation

This validation query checks the two parts of the UNION ALL separately. It counts how many countries satisfy the high-GDP condition and how many countries satisfy the high-CO_2 condition. These two counts should correspond to the number of rows generated by each category in the main query. This validates that the combined result is not mixing the two logics incorrectly and that each category contributes its own rows.

```
SELECT 'High GDP' AS "Category", COUNT(*) AS "Total Rows"
FROM Yearly_Stats ys
WHERE ys.year_number = 2015 AND ys.country_name != 'World' AND ys.gdp > (
    SELECT AVG(ys2.gdp)
    FROM Yearly_Stats ys2
    WHERE ys2.year_number = 2015 AND ys2.country_name != 'World')
UNION ALL
SELECT 'High CO2 Emission' AS "Category", COUNT(*) AS "Total Rows"
FROM Yearly_Energy_Records yer
WHERE yer.year_number = 2015 AND yer.type_name = 'all_energy_types' AND yer.country_name
!= 'World' AND yer.co2_emission > (
    SELECT AVG(yer2.co2_emission)
    FROM Yearly_Energy_Records yer2
    WHERE yer2.year_number = 2015 AND yer2.type_name = 'all_energy_types' AND
yer2.country_name != 'World');
```

	CATEGORY	TOTAL ROWS
1	High GDP	34
2	High CO2 Emission	33

7.30 (Which countries have no government policy recorded)

```sql
SELECT c.country_name
FROM Country c
WHERE c.country_name NOT IN (
    SELECT DISTINCT gp.country_name
    FROM Government_Policy gp
)
AND c.country_name NOT IN (
    'World', 'Africa', 'Asia', 'Europe', 'European Union (27)',
    'North America', 'South America', 'Oceania', 'Antarctica',
    'Former U.S.S.R.', 'Former Yugoslavia', 'Former Czechoslovakia',
    'Former Serbia and Montenegro'
)
ORDER BY c.country_name;
```

	COUNTRY_NAME
1	American Samoa
2	Anguilla
3	Antarctica (the ter...
4	Bouvet Island (Bo...
5	British Indian Oce...
6	British Virgin Islands
7	Brunei Darussalam
8	Burma
9	Cabo Verde
10	Cape Verde
11	Cayman Islands
12	Christmas Island

7.30.1 Explanation

This query identifies countries that have no associated government policy in the database. A NOT IN subquery extracts all country names that appear at least once in Government_Policy, and the outer query returns all countries from the Country table that are absent from that set. This approach is straightforward and avoids unnecessary joins.

7.30.2 Validation

The validation query uses a LEFT JOIN between Country and Government_Policy, grouping by country and filtering with HAVING COUNT(policy_id) = 0. Any country with no matching policy will have a count of zero. Both queries must return the exact same set of countries for the result to be valid.

```
SELECT c.country_name, COUNT(gp.policy_id) AS total_policies

FROM Country c

LEFT JOIN Government_Policy gp ON c.country_name = gp.country_name

WHERE c.country_name NOT IN (

    'World', 'Africa', 'Asia', 'Europe', 'European Union (27)',

    'North America', 'South America', 'Oceania', 'Antarctica',

    'Former U.S.S.R.', 'Former Yugoslavia', 'Former Czechoslovakia',

    'Former Serbia and Montenegro'

)

GROUP BY c.country_name

HAVING COUNT(gp.policy_id) = 0

ORDER BY c.country_name;
```

	COUNTRY_NAME	TOTAL_POLICIES
1	American Samoa	0
2	Anguilla	0
3	Antarctica (the ter...	0
4	Bouvet Island (Bo...	0
5	British Indian Oce...	0
6	British Virgin Islands	0
7	Brunei Darussalam	0
8	Burma	0
9	Cabo Verde	0
10	Cape Verde	0
11	Cayman Islands	0
12	Christmas Island	0
13	Cocos (Keeling) Isl...	0
14	Congo-Brazzaville	0
15	Congo-Kinshasa	0
16	Cook Islands	0

7.31 Trigger 1

7.31.1 Introduction

The primary objective of this trigger is to establish an automated mechanism for critical infrastructure records. Specifically, it monitors the Power_Plant table for any modifications made to the power_plant_status attribute. Upon detecting a status change (e.g., from 'Operational' to 'Decommissioned'), the trigger automatically captures the historical state and inserts a permanent log entry into a dedicated supplementary audit table. This ensures traceability and environmental compliance tracking without requiring manual data entry from the application layer.

7.31.2 Solution

```
-- 1. Creation of the supplementary audit table
CREATE TABLE Power_Plant_Audit (
    power_plant_name VARCHAR2(255),
    old_status VARCHAR2(100),
    new_status VARCHAR2(100),
    change_date DATE
);
-- 2. Creation of the Trigger
CREATE TRIGGER trg_audit_plant_status
AFTER UPDATE OF power_plant_status ON Power_Plant
FOR EACH ROW
WHEN (OLD.power_plant_status != NEW.power_plant_status)
BEGIN
    INSERT INTO Power_Plant_Audit (
        power_plant_name,
        old_status,
        new_status,
        change_date
    )
    VALUES (
        :OLD.power_plant_name,
        :OLD.power_plant_status,
        :NEW.power_plant_status,
        SYSDATE
    );
END;
```

7.31.3 Explanation

The trg_audit_plant_status is an AFTER UPDATE trigger specifically designed to monitor modifications to the power_plant_status attribute within the Power_Plant table. A critical feature is the WHEN (OLD.power_plant_status != NEW.power_plant_status) conditional clause, which ensures the trigger's execution block only fires if a genuine state change has occurred, effectively preventing the generation of redundant logs if an update merely rewrites the existing status.

Once triggered, the BEGIN...END block executes an INSERT statement to populate the supplementary Power_Plant_Audit table. It utilizes the :OLD and :NEW bind variables within the VALUES clause to accurately capture the exact pre-transaction plant name and status alongside the newly updated status, permanently stamping the historical record with the database's current system date (SYSDATE).

The colons (:OLD and :NEW) are mandatory in the VALUES section because they act as bind variables, telling the database to fetch the exact temporary values held in memory during the transaction rather than treating them as standard static column names.

7.31.4 Validation

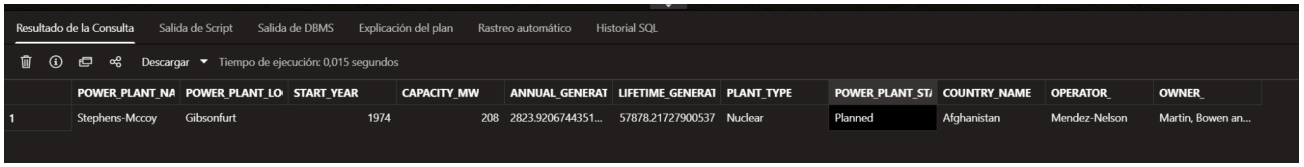
To substantiate the assertion that the trigger behaves congruently with its intended design, a suite of data manipulation language tests was executed.

Test 1: The Valid UPDATE Test.

Objective: prove the trigger accurately captures a status change.

```
SELECT * FROM POWER_PLANT WHERE POWER_PLANT_NAME = 'Stephens-Mccoy';
```

Initially, status: Planned.



Then:

```
-- Action: Modify the operational status of an existing plant.

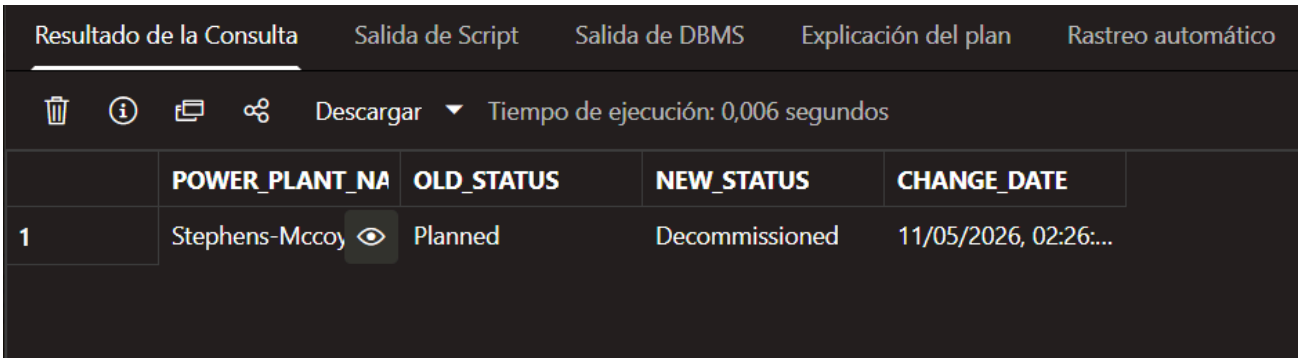
UPDATE Power_Plant

SET power_plant_status = 'Decommissioned'

WHERE power_plant_name = 'Stephens-Mccoy';
```

```
-- Validation Query:

SELECT * FROM Power_Plant_Audit WHERE power_plant_name = 'Stephens-Mccoy';
```



Result: The query successfully returns a new row detailing the old status, the new 'Decommissioned' status, and the exact timestamp. This proves the core functionality is active

7.32 Trigger 2

7.32.1 Introduction

The primary objective of this trigger is to establish an automated monitoring mechanism for government policy budgets. Specifically, it monitors the Government_Policy table for any new insertions. Upon detecting a new policy that simultaneously exceeds the current average budget across all existing policies AND has a target reduction percentage above 20%, the trigger automatically captures the relevant information and inserts a permanent alert entry into a dedicated supplementary table Policy_Budget_Alert. Additionally, a country-level counter is maintained to track how many high-impact alerts have been generated per country, providing an extra layer of analytical traceability.

7.32.2 Solution

```
-- 1. Creation of the supplementary alert table
CREATE TABLE Policy_Budget_Alert (
    policy_id                NUMBER,
    name_policy              VARCHAR2(255),
    budget_million_usd       NUMBER,
    avg_budget_at_insert     NUMBER,
    target_reduction_percent NUMBER,
    alert_date               DATE,
    country_name             VARCHAR2(255),
    country_alert_count      NUMBER
);

-- 2. Creation of the Trigger
CREATE OR REPLACE TRIGGER trg_policy_budget_alert
BEFORE INSERT ON Government_Policy
FOR EACH ROW
DECLARE
    v_avg_budget  NUMBER;
    v_alert_count NUMBER;
BEGIN
    SELECT AVG(budget_million_usd)
    INTO v_avg_budget
    FROM Government_Policy;

    IF :NEW.budget_million_usd > v_avg_budget
    AND :NEW.target_reduction_percent > 20 THEN
```

```
        SELECT COUNT(*) + 1
        INTO v_alert_count
        FROM Policy_Budget_Alert
        WHERE country_name = :NEW.country_name;

        INSERT INTO Policy_Budget_Alert (
            policy_id,
            name_policy,
            budget_million_usd,
            avg_budget_at_insert,
            target_reduction_percent,
            alert_date,
            country_name,
            country_alert_count
        )
        VALUES (
            :NEW.policy_id,
            :NEW.name_policy,
            :NEW.budget_million_usd,
            v_avg_budget,
            :NEW.target_reduction_percent,
            SYSDATE,
            :NEW.country_name,
            v_alert_count
        );
    END IF;
END;
```

7.32.3 Explanation

The trg_policy_budget_alert is a BEFORE INSERT trigger designed to monitor new entries in the Government_Policy table. A BEFORE INSERT trigger is used instead of AFTER INSERT to avoid the ORA-04091 mutating table error since the new record has not yet been committed to the table, the SELECT AVG can safely read existing rows without conflict.

Upon each insertion, the DECLARE block initializes two local variables: v_avg_budget, which dynamically computes the current average budget across all existing policies, and v_alert_count, which tracks how many alerts have already been generated for the same country. The IF conditional

enforces two simultaneous conditions the new policy's budget must strictly exceed the average AND its target_reduction_percent must be greater than 20%. Only when both conditions are met does the trigger fire the INSERT into Policy_Budget_Alert. The :NEW bind variables capture the exact values of the incoming record, SYSDATE permanently stamps the alert with the current system date, and v_alert_count is incremented by 1 for each new alert per country.

7.32.4 Validation

Test 1: Insert a policy with budget below the average — no alert should be generated.

```
INSERT INTO Government_Policy (policy_id, name_policy, budget_million_usd,
target_reduction_percent, policy_status, start_policy_year, policy_type, country_name)

VALUES (9999, 'Test Policy Low', 10, 5, 'Active', 2020, 'Fiscal Incentive', 'Spain');



-- Validation: should return no rows

SELECT * FROM Policy_Budget_Alert WHERE policy_id = 9999;
```

POLICY_ID	NAME_POLICY	BUDGET_MILLION_	AVG_BUDGET_AT_II	TARGET_REDUCTIO	ALERT_DATE	COUNTRY_NAME	COUNTRY_ALERT_C
No se ha encontrado ningún dato							

Test 2 Insert a policy with budget above average but target below 20% — no alert should be generated.

```


INSERT INTO Government_Policy (policy_id, name_policy, budget_million_usd,
target_reduction_percent, policy_status, start_policy_year, policy_type, country_name)


VALUES (10000, 'Test Policy High Low Target', 999999, 5, 'Active', 2021, 'Emissions
Reduction', 'Spain');


-- Validation: should return no rows


SELECT * FROM Policy_Budget_Alert WHERE policy_id = 10000;
```

POLICY_ID	NAME_POLICY	BUDGET_MILLION_	AVG_BUDGET_AT_II	TARGET_REDUCTIO	ALERT_DATE	COUNTRY_NAME	COUNTRY_ALERT_C
No se ha encontrado ningún dato							

Test 3 Insert a policy with budget above average AND target above 20% — alert should be generated with country_alert_count = 1. .

```
INSERT INTO Government_Policy (policy_id, name_policy, budget_million_usd,
target_reduction_percent, policy_status, start_policy_year, policy_type, country_name)

VALUES (10003, 'Test Policy High', 999999, 50, 'Active', 2021, 'Emissions Reduction',
'Spain');



-- Validation: should return one row with country_alert_count = 1

SELECT * FROM Policy_Budget_Alert WHERE policy_id = 10003;
```

	POLICY_ID	NAME_POLICY	BUDGET_MILLION_	AVG_BUDGET_AT_II	TARGET_REDUCTIO	ALERT_DATE	COUNTRY_NAME	COUNTRY_ALERT_C
1	10003	Test Policy High	999999	5484.3117529880...	50	17/05/2026, 04:12:...	Spain	1

Test 4 Insert a second high-impact policy for the same country — country_alert_count should increment to 2.

```
INSERT INTO Government_Policy (policy_id, name_policy, budget_million_usd,
target_reduction_percent, policy_status, start_policy_year, policy_type, country_name)

VALUES (10004, 'Test Policy High 2', 999999, 50, 'Active', 2022, 'Emissions Reduction',
'Spain');



-- Validation: should return two rows for Spain, with country_alert_count = 1 and 2

SELECT * FROM Policy_Budget_Alert WHERE country_name = 'Spain' ORDER BY alert_date;
```

	POLICY_ID	NAME_POLICY	BUDGET_MILLION_	AVG_BUDGET_AT_II	TARGET_REDUCTIO	ALERT_DATE	COUNTRY_NAME	COUNTRY_ALERT_C
1	10003	Test Policy High	999999	5484.3117529880...	50	17/05/2026, 04:12:...	Spain	1
2	10004	Test Policy High 2	999999	6473.8786069651...	50	17/05/2026, 04:14:...	Spain	2

7.33. Trigger 3

7.33.1 Introduction

The objective of this trigger is to automatically classify environmental pollution records according to their severity level. When a new row is inserted into the Registers table, the trigger evaluates the pollution indicators and automatically assigns a pollution category into a supplementary table called Environmental_Classification.

7.33.2 Solution

```
CREATE TABLE Environmental_Classification (
    classification_id INT GENERATED BY DEFAULT AS IDENTITY,
    city_name VARCHAR2(100),
    country_name VARCHAR2(100),
    day_number INT,
    month_number INT,
    year_number INT,
    pollution_level VARCHAR2(50),
    classification_date DATE,
    CONSTRAINT pk_environmental_classification PRIMARY KEY (classification_id)
);



CREATE TRIGGER trg_environmental_classification
AFTER INSERT ON Registers
FOR EACH ROW
DECLARE
    v_pollution_level VARCHAR2(50);
BEGIN
    IF :NEW.pm10 >= 150 OR :NEW.no2 >= 100 OR :NEW.o3 >= 180 THEN
        v_pollution_level := 'Critical Pollution';
    ELSIF :NEW.pm10 >= 80 OR :NEW.no2 >= 50 OR :NEW.o3 >= 120 THEN
        v_pollution_level := 'Moderate Pollution';
    ELSE
        v_pollution_level := 'Low Pollution';
    END IF;

    INSERT INTO Environmental_Classification (
        city_name,
        country_name,
```

```
        day_number,
        month_number,
        year_number,
        pollution_level,
        classification_date
    )
    VALUES (
        :NEW.city_name,
        :NEW.country_name,
        :NEW.day_number,
        :NEW.month_number,
        :NEW.year_number,
        v_pollution_level,
        SYSDATE
    );
END;
```

7.33.3 Explanation

The trg_environmental_classification trigger is executed after a new record is inserted into the Registers table. It analyzes the pollution values pm10, no2, and o3 and assigns a pollution category depending on the thresholds defined in the trigger. The result is then stored in Environmental_Classification, allowing the database to automatically enrich raw pollution data with a meaningful classification.

7.33.4 Validation

```
INSERT INTO Registers (city_name, country_name, day_number, month_number, year_number, pm10, no2, o3)
VALUES ('Bangkok', 'Thailand', 26, 4, 2023, 30.0, 20.0, 50.0);


INSERT INTO Registers (city_name, country_name, day_number, month_number, year_number, pm10, no2, o3)
VALUES ('Bangkok', 'Thailand', 18, 6, 2023, 90.0, 40.0, 80.0);


INSERT INTO Registers (city_name, country_name, day_number, month_number, year_number, pm10, no2, o3)
VALUES ('Bangkok', 'Thailand', 15, 5, 2023, 200.0, 120.0, 250.0);
```

```
SELECT city_name, country_name, day_number, month_number, year_number, pollution_level,
classification_date

FROM Environmental_Classification
```

Query Result | Script Output | DBMS Output | Explain Plan | Autotrace | SQL History

Download | Execution time: 0.006 seconds

	CITY_NAME	COUNTRY_NAME	DAY_NUMBER	MONTH_NUMBER	YEAR_NUMBER	POLLUTION_LEVEL	CLASSIFICATION_D
1	Bangkok	Thailand	18	6	2023	Moderate Pollution	05/14/2026, 05:33:...
2	Bangkok	Thailand	26	4	2023	Low Pollution	05/14/2026, 05:32:...
3	Bangkok	Thailand	15	5	2023	Critical Pollution	05/14/2026, 05:34:...

To validate the trigger, three different records were inserted into the Registers table using valid city, country, and date values from the database. Each record contains different pollution levels for pm10, no2, and o3. After each insertion, the Environmental_Classification table was queried to verify that the trigger had automatically generated the correct classification. The first test produced a Low Pollution result, the second one produced a Moderate Pollution result, and the third one produced a Critical Pollution result. This confirms that the trigger correctly evaluates the pollution thresholds and enriches the data automatically after each new insertion.

8 Conclusions

8.1 Utilization of resources (1 page)

Phase	Jan Porcar	Francesc Mateu	Giovanni Luca	Total
Entity-Relationship model	3h	2h	5h	10h
Relational model	3h	3h	3h	9h
Datatype selection	2h	2h	2h	6h
Physical model codification	3h	4h	2h	9h
Data import	2h	3h	1h	6h
Query and trigger implementation	3h	4h	2h	9h
Report	2h	1h	4h	7h
Total:	18h	19h	19h	56h

The table illustrates a highly equitable overall distribution of labor, with total individual contributions ranging between 18 and 19 hours, culminating in 56 total project hours. Phases requiring strict architectural consensus, specifically the Relational Model (9h) and Datatype Selection (6h), exhibit zero deviation; all three team members contributed equal hours to ensure the database's structural integrity was correctly developed.

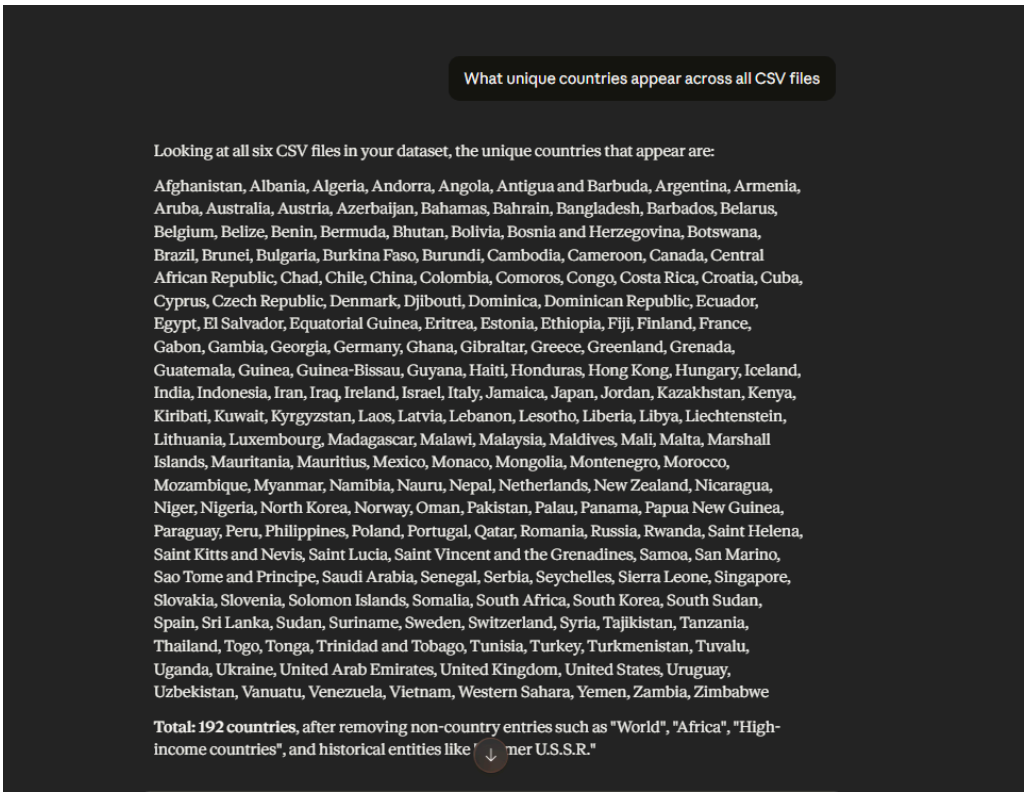
Jan Porcar maintained a highly consistent baseline of hours across the entire project.The notable deviations in the remaining phases reflect a strategic division of labor based on specific project needs and technical specialization. Giovanni Luca assumed a heavier workload during the foundational Entity-Relationship Model phase (5h) and the final Report redaction (4h), effectively framing the project's conceptual beginning and its formal documentation. Conversely, Francesc Mateu absorbed the bulk of the complex database mechanics. He logged the highest hours during Physical Model Codification (4h), Data Import (3h), and Query/Trigger Implementation (4h). This deviation accounts for the extra time required to resolve intricate Oracle constraints, manage staging table logic for dirty CSV files, and debug complex SQL syntax.

8.2 AI usage

-**Prompt 1**: "What unique countries appear across all CSV files?"

Rationale: We needed a clean country list for the Mockaroo custom field without manually checking 900,000 rows.

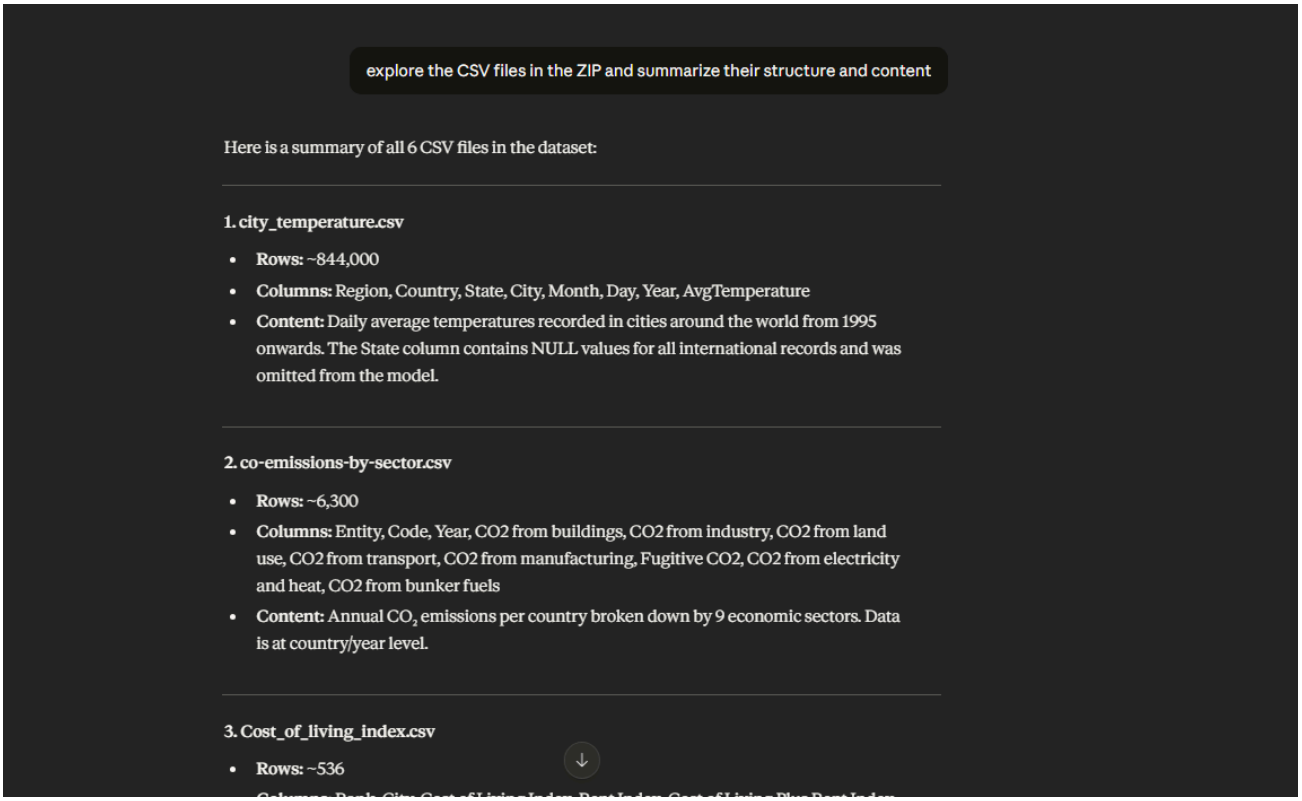
Result: A cleaned list of 192 countries was obtained and used directly in the Mockaroo configuration.



-**Prompt 2:** "Explore the CSV files in the ZIP and summarize their structure and content."

Rationale: Before starting the conceptual design, we needed a quick overview of the structure, number of rows, and content of each file to understand what entities and relationships would be needed.

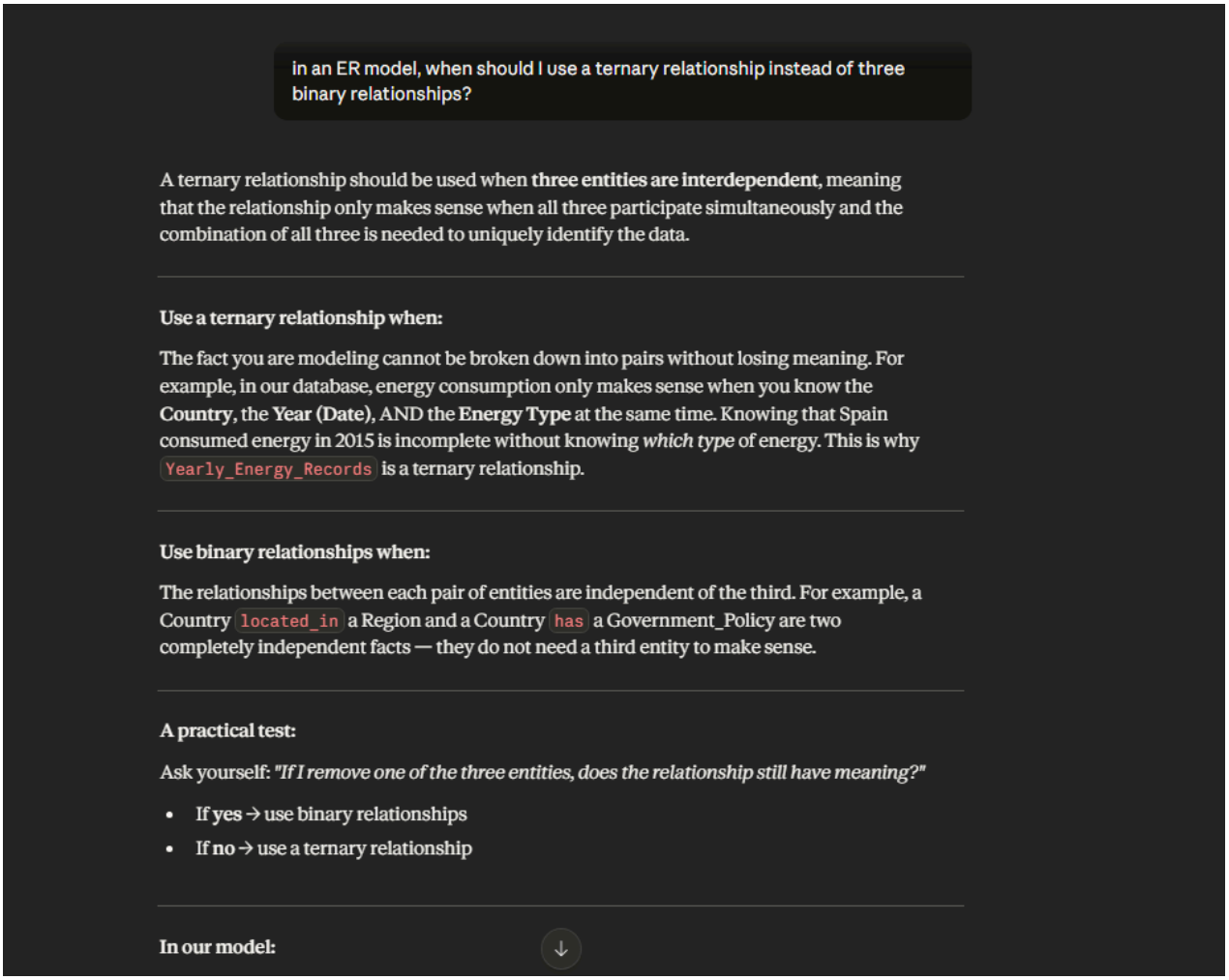
Result: Claude provided a structured summary of all 6 files including column names, row counts, and a description of the content of each file. This helped the team identify key entities such as Country, City, Power_Plant, and Energy_Type before starting the ER diagram in Miro.



Prompt 3: "In an ER model, when should I use a ternary relationship instead of three binary relationships?"

Rationale: We were unsure whether to model the energy data as a ternary relationship or as separate binary relationships between Country, Date, and Energy_Type.

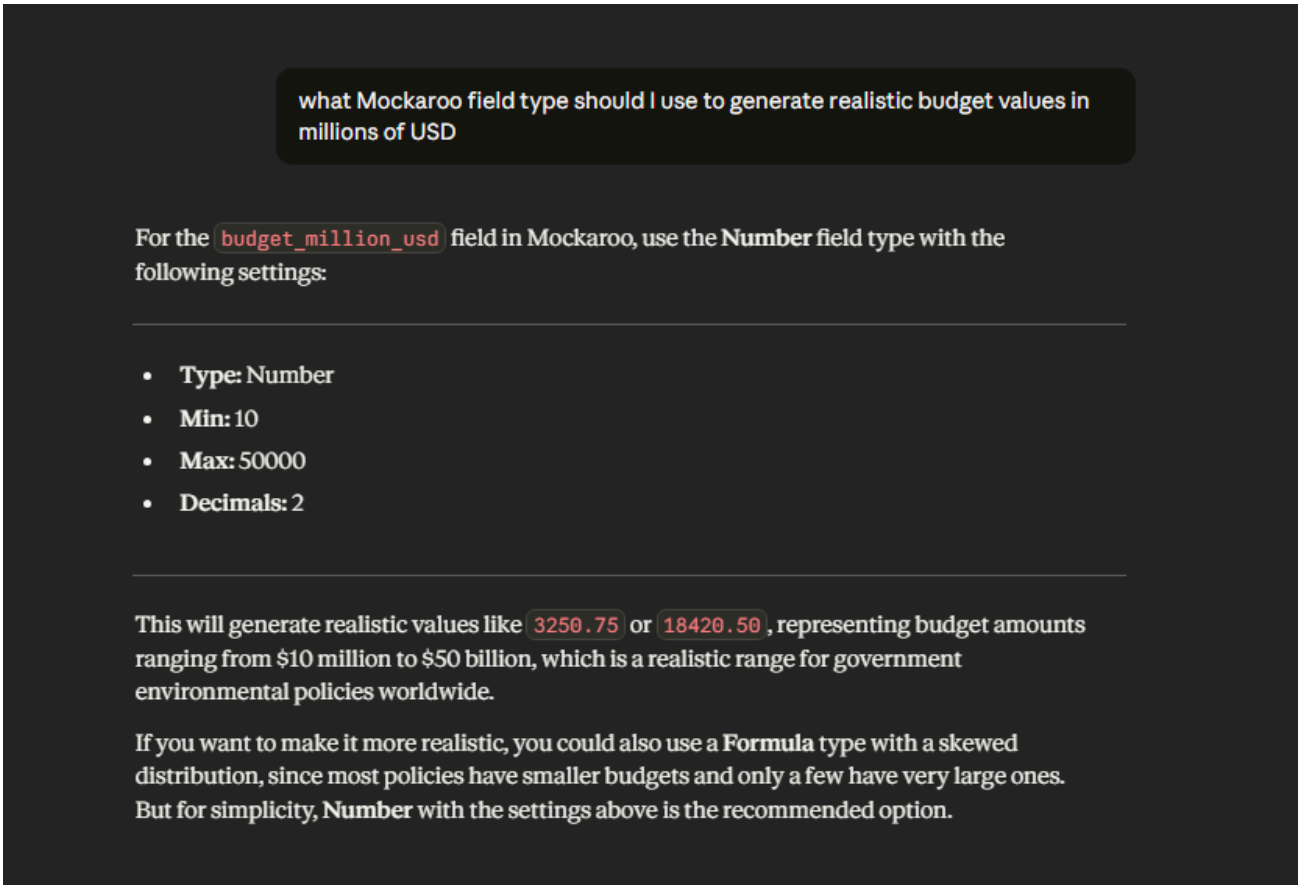
Result: Claude explained that a ternary relationship is appropriate when the three entities are interdependent and the data only makes sense when all three participate simultaneously. This confirmed the team's decision to model Yearly_Energy_Records as a ternary relationship, since energy consumption is only uniquely identifiable when Country, Date, and Energy_Type are all known at the same time.



-**Prompt 4:** "what Mockaroo field type should I use to generate realistic budget values in millions of USD?"

Rationale: we were unsure which Mockaroo field type and configuration to use for the budget_million_usd column in the government_policy.csv file to generate realistic and meaningful data.

Result: Claude recommended using the Number field type with a minimum of 10, maximum of 50,000, and 2 decimal places. This generates values ranging from $10 million to $50 billion, which is a realistic range for government environmental policies worldwide. The team applied this configuration directly in Mockaroo before generating the dataset.



-Prompt 5: "can you check if this validation question can be answered with my current ER model: Which countries have active emissions reduction policies but are also in the top 10 for total CO_2 emissions in 2015?"

Rationale: after drafting the 30 validation questions, we wanted to verify that each question could actually be answered using the entities and relationships defined in the ER model, to avoid questions that would be impossible to implement in Stage 3.

Result: Claude reviewed the question against the ER model and confirmed that it can be answered by joining the Government_Policy entity (filtering by status = "Active" and policy_type = "Emissions Reduction") with the yearly_emits relationship (filtering by year = 2015 and ranking countries by total CO_2). The team took note of this for the data import phase.

can you check if this validation question can be answered with my current ER model: Which countries have active emissions reduction policies but are also in the top 10 for total CO_2 emissions in 2015

Yes, this question can be answered with your current ER model. Here is how:

Entities/relationships needed:

- `Government_Policy` → filter by `status = "Active"` and `policy_type = "Emissions Reduction"`
- `Country` → to link both sides
- `yearly_emits` → filter by `year = 2015`, sum all CO_2 sectors, rank countries by total

---

How the query would work:

1. Get all countries that have at least one active emissions reduction policy from `Government_Policy`
2. Calculate the total CO_2 emissions per country in 2015 by summing all 9 sectors from `yearly_emits`
3. Rank countries by total CO_2 and keep only the top 10
4. Find the intersection — countries that appear in both lists

---

Potential issue to watch out for:

Country names must be **consistent** between `government_policy.csv` (generated with Mockaroo) and `co-emissions-by-sector.csv` (original dataset). For example, if Mockaroo generates "United States" but the emissions file uses "USA", the JOIN will fail in Stage 3. The team should ensure name consistency during the data import phase.

8.3 Lessons learned and conclusions

The culmination of this project has reshaped our understanding of database management, successfully bridging the gap between theoretical relational model and the physical realities of enterprise data architecture. Initially, designing the Entity-Relationship model felt like an abstract exercise in conceptual logic. However, translating that framework into a rigorous relational schema demonstrated exactly why normalization rules up to the Third Normal Form are non-negotiable in professional environments. We learned that architectural decisions made in the earliest design phases such as elevating the temporal dimension into isolated year and exact date entities, or decoupling demographic statistics from granular energy records have massive cascading effects on both query performance and absolute data integrity.

Furthermore, the implementation of the validation queries and automated triggers significantly advanced our technical proficiency in SQL. Moving far beyond basic data retrieval commands, we learned to leverage advanced grouping, dynamic scalar subqueries, and complex mathematical aggregations to extract meaningful environmental insights from hundreds of thousands of raw data points. Designing active database logic to autonomously audit infrastructure state changes and monitor government policy budgets taught us how to embed critical business rules directly into the storage layer. This ensures environmental compliance and tracking without relying on external application logic. Navigating Oracle's strict syntax, such as the mandatory use of variables to capture temporary transactional memory, forced us to adopt an analytical approach to database programming.

Ultimately, this project reinforced the critical importance of architectural and collaborative software engineering. Distributing the database workload required continuous communication and strict adherence to our agreed-upon physical model to ensure our individual table structures synchronized flawlessly during the final database deployment. The successful transition from scattered, unformatted climate and energy spreadsheets into a fully functional, highly relational Oracle database stands as a testament to the power of structured data modeling. We conclude this subject not merely with a theoretical understanding of relational databases, but with the practical, hard-earned capability to design, deploy, and interrogate complex, real-world data.